

一、基础

模拟面试

Transformer

Q1: 请你整体介绍一下 Transformer 架构，以及 Self-Attention 的核心作用

Transformer 主要由 Encoder 和 Decoder 组成（原始论文结构），每一层包括 Multi-Head Self-Attention 和 Feed Forward Network。

Self-Attention 的核心是通过 Q、K、V 计算序列中每个 token 与其他 token 的相关性，从而实现全局依赖建模。它解决了 RNN 难以并行计算以及长距离依赖难以捕捉的问题。

Q2: 为什么 Self-Attention 可以解决“长距离依赖”，RNN 的瓶颈本质是什么？

RNN 是链式结构，信息需要逐步传递，路径长度是 $O(n)$ ，导致远距离 token 梯度容易消失。

Self-Attention 中任意两个 token 直接计算相关性，路径长度是 $O(1)$ ，因此更容易建模长距离依赖。

Q3: Self-Attention 计算复杂度是 $O(n^2)$ ，为什么工业界还能用？

虽然计算复杂度是 $O(n^2)$ ，但 Transformer 支持高度并行化，GPU 上效率高。

同时在工程上可以通过：

sparse attention
sliding window attention
FlashAttention
等方式优化。

Q4: 你提到了 FlashAttention，它具体优化了什么瓶颈？

FlashAttention 主要优化的是 显存 IO（HBM 访问），不是数学复杂度。

通过 tile 分块计算 attention，避免显式存储完整 attention matrix，从而减少显存读写，提高速度。

Q5: 如果不用 FlashAttention，标准 attention 的 memory bottleneck（内存瓶颈）在哪里？

主要瓶颈是：

Attention matrix 是 $n \times n$
需要存储 $\text{softmax}(QK^T)$
反向传播还要保存中间结果

所以显存占用是 $O(n^2)$ ，长序列会直接爆显存。

Q6: Transformer 没有 RNN 的顺序结构，那它是怎么建模 token 顺序的？

通过 Positional Encoding（位置编码）注入位置信息。

原始 Transformer 使用 sin/cos 位置编码，也可以学习 embedding。

Q7: 有哪些位置编码？

Q8: 为什么 sin/cos 位置编码是“可外推”的？相比 learned embedding 优势是什么？

sin/cos 是函数形式，可以泛化到训练长度之外的 position。

而 learned embedding 只能覆盖训练时见过的位置，超长序列会失效。

Q9: 现在 LLM 为什么很多不用 absolute positional encoding，而用 RoPE？

RoPE（Rotary Position Embedding）是相对位置编码的一种实现方式。

它的优势：

将 position 以旋转方式作用在 QK 上
attention score 天然包含相对距离
更适合长上下文建模

Q10: RoPE 在长上下文外推时会有什么问题？

会出现 频率外推失真（extrapolation degradation），因为旋转角度线性增长后超出训练分布。

因此会有：

NTK scaling
YaRN
等方法做修正。

Q11: 推理阶段 Transformer 最大的工程优化点是什么？为什么？

主要是 KV Cache。

因为自回归生成时：

K、V 可以复用历史 token
不需要重复计算
将复杂度从 $O(n^2)$ 降为 $O(n)$

Q12: KV Cache 可以优化推理，那为什么不能用于训练？

KV Cache 是在自回归推理时缓存历史 K/V，避免重复计算。

训练阶段是并行计算整个序列，每个 token 都要 attend 所有 token，所以无法复用历史 cache。

Q13: 训练也是可以 cache 的，你知道哪里吗？

训练时：

forward 是全序列 attention

但 decoder-only 模型在 causal mask 下是可以做 block-wise KV reuse 的（如 FlashAttention2 / sequence packing）

但问题是：

反向传播需要完整 attention graph

KV cache 会破坏 computation graph 的完整性

Q14: 那为什么 inference 可以 cache，training 不行？本质差异到底是什么？

Training: 需要梯度传播 + 完整 attention matrix

Inference: 只是 forward sampling

KV Cache 本质：

用空间换时间，但破坏了反向传播依赖结构

Q15: 你说 attention 是 $QK^T / \sqrt{d}$ ，为什么一定要除 $\sqrt{d}$ ？

防止 dot product 结果过大导致 softmax saturation。

Q16: 如果不用 scaling，会发生什么“数学级别”的问题？

如果不缩放：

QK^T 方差随 d 增大

softmax 输入进入极端区间

导致：

梯度接近 0

attention 变成 one-hot

Q17: 那为什么 scaling 用的是 $\sqrt{d}$ ，而不是 d 或 $\log d$ ？

Q18: Transformer 为什么一定要 Multi-Head？单头不行吗？

Multi-head 可以让模型在不同子空间学习不同关系。

本质区别：

Multi-head = 多个低维 attention subspace

Single-head = 一个高维 attention space

Q19: 那如果我把 single-head dimension 调到 multi-head 总维度一样, 理论上表达能力是否等价?

理论上表达能力接近, 但优化性质不同:

Multi-head 优势在:

更容易优化 (inductive bias)

梯度更稳定

不同 head 专注不同 pattern

Q20: Transformer 为什么需要 residual connection? 去掉会怎样?

避免梯度消失, 使训练稳定。

因为深层 MLP + attention stack:

深层矩阵乘法链

softmax + linear projection

Jacobian 连乘

仍然可能:

gradient norm exponential decay

Q21: Residual connection 本质上解决的是什么数学问题?

本质是:

让网络学习 residual function:

$$F(x) = x + g(x)$$

等价于优化:

identity mapping easier to preserve

reduces function space curvature

improves conditioning of optimization landscape

Q22: 正向传播推导

设输入矩阵为

$$X \in R^{N \times d}$$

其中 N 为序列长度 (Sequence Length), d 为特征维度 (Embedding Dimension)。

定义以下权重矩阵:

- 查询投影矩阵:

$$W_Q \in R^{d \times d_k}$$

- 键投影矩阵:

$$W_K \in R^{d \times d_k}$$

- 值投影矩阵:

$$W_V \in R^{d \times d_k}$$

- 输出投影矩阵:

$$W_O \in R^{d \times d_k}$$

自注意力机制的正向传播可以分为以下五个步骤：

Step 1: 线性投影 (Linear Projection)

$$Q = XW_Q, K = XW_K, V = XW_V$$

Step 2: 计算注意力得分 (Attention Scores)

计算 Q 和 K 的内积，并进行缩放：

$$S = QK^T / \sqrt{d}$$

Step 3: Softmax 归一化 (Attention Weights)

对得分矩阵的每一行进行 Softmax 归一化，得到概率分布矩阵 P：

$$P = softmax(S)$$

其中，对于矩阵的第 i 行、第 j 列：

$$P_{ij} = e^{S_{ij}} / \sum_{m=1}^N e^{S_{im}}$$

Step 4: 加权求和 (Context Vector)

利用权重矩阵 P 对 V 进行加权求和，得到自注意力的直接输出 O：

$$O = PV$$

Step 5: 输出线性变换 (Output Projection)

经过最后的线性层映射回原始维度：

$$Y = OW_O$$

Q22: 手撕 Transformer 反向传播推导 (核心版)

Step 1: Attention forward

$$Q = XW_Q, K = XW_K, V = XW_V$$

$$A = softmax(QK^T / \sqrt{d})$$

$$O = AV$$

Step 2: 定义 loss

Step 3: 从输出开始反传

Step 4: softmax 求导 (关键难点)

Step 5: 对 Z 求梯度

Step 6: 对 Q 和 K

Step 7: 回到 W_Q, W_K, W_V

Transformer 反向传播核心是：

“softmax Jacobian + QK bilinear form + matrix chain rule”

Q23: Self-Attention 直观理解

Self-Attention（自注意力）就是让序列里的每一个词，都能“环顾四周”，根据上下文的重要性，动态地从所有其他词那里“抽取”信息来丰富自己，从而生成更深层的语义表示。

Q24: Transformer 整体结构

Transformer 本质上是一个基于编码器-解码器（**Encoder-Decoder**）架构的序列模型，但它彻底抛弃了传统的 **RNN/CNN**，完全依赖注意力机制来捕捉全局依赖关系。整个结构可以看作一个沙漏型：左边是编码器，右边是解码器，中间通过交叉注意力相连。

1. 输入端（模型的“五官”）

输入先经过 **Embedding** 层转为向量。

关键点：由于 **Self-Attention** 没有顺序感（它是置换不变的），必须加上 **位置编码（Positional Encoding）**。原论文用的正弦余弦，现在很大模型用可学习的位置编码或 **RoPE**（旋转位置编码）。

2. 编码器（Encoder，负责“阅读理解”）

它由 N 个（原论文 6 个）完全相同的层堆叠而成，每一层有两个核心子层：

子层1（多头自注意力）：每个词看全局，动态聚合上下文信息（这就是你刚才问的 **Self-Attention** 的具体落地）。

子层2（前馈神经网络 **FFN**）：一个两层的全连接网络（先升维再降维），负责对每个位置的向量做非线性变换，提炼特征。

关键设计：每个子层外面都套了残差连接（**Add**）和层归一化（**Norm**）。顺序是 先 **Attention/FFN**，再 **Add**，再 **Norm**（即 **Post-Norm**）或现在流行的 **Pre-Norm**（先 **Norm** 再操作，训练更稳）。

3. 解码器（Decoder，负责“逐词翻译/生成”）

它也由 N 个层堆叠，但每一层比编码器多了一个子层（共3个），且做了遮罩：

子层1（带掩码的自注意力）：和编码器一样，但加了 **Mask**（掩码）。因为在生成第 t 个词时，不能偷看未来的词（即 $t+1, t+2, \dots$ ），所以要遮住右边未生成的部分。

子层2（交叉注意力 / **Encoder-Decoder Attention**）：这是连接编码器和解码器的桥梁。它的 **Q**（查询）来自解码器上一个子层的输出，而 **K**（键）和 **V**（值）来自编码器的最终输出。直观理解就是：解码器每生成一个词，都要去编码器那边“查字典”，看源文本中哪些词最相关。

子层3（前馈神经网络 **FFN**）：同上，做特征强化。

4. 输出端（模型的“嘴巴”）

解码器最后输出的向量，经过一个 **线性层（Linear）** 映射到词表大小，再套一个 **Softmax** 输出概率分布，挑概率最高的词作为预测结果。

Q25: Post-Norm 和 Pre-Norm 有什么区别？

第一步：一句话定差异（面试官第一印象）

Post-Norm 和 **Pre-Norm** 的核心区别在于层归一化（**Layer Norm**）在残差块中的位置不同。

Post-Norm（原始Transformer）：先做子层操作，再做归一化。公式是 $\text{Norm}(x + \text{SubLayer}(x))$ 。

Pre-Norm（现代主流）：先做归一化，再做子层操作。公式是 $x + \text{SubLayer}(\text{Norm}(x))$ 。

别看只是调换了顺序，这直接导致了深层网络训练难度的天壤之别。

第二步：用公式和“梯度流”讲透本质（核心技术点）

1. Post-Norm（原始设计，理想很丰满）

公式：输出 = LayerNorm($x + \text{SubLayer}(x)$)

致命问题（为什么难训）：因为 LayerNorm 是整个残差块的“最后一道门”。在反向传播时，梯度必须穿过 LayerNorm 层才能流回 SubLayer。

后果：LayerNorm 会对输入进行缩放和平移，导致靠近输出层的梯度在回传时被大幅压缩或扭曲。尤其在深层（比如 12 层以上），浅层（靠近输入层）的梯度极易爆炸或消失。所以原始 Transformer 必须配合学习率 warmup（先小后大）来勉强稳定训练——warmup 本质上就是在等 Post-Norm 的梯度方差逐渐稳定下来。

2. Pre-Norm（现代改良，现实很骨感）

公式：输出 = $x + \text{SubLayer}(\text{Norm}(x))$

核心优势（为什么稳）：残差连接 $x + \dots$ 把 LayerNorm “隔离”在了子层内部。反向传播时，梯度走两条高速公路：

路径一（捷径）：梯度直接通过 x 恒等映射回传，完全不经过任何归一化层，完好无损地传到浅层。

路径二（子层）：梯度才需要穿过 Norm 和 SubLayer。

结论：总有“一条梯度高速公路”是绝对通畅的。这就保证了即使堆叠上千层，浅层也能收到高质量的信号，无需 warmup 也能稳定训练。

第三步：权衡取舍（展现辩证思维，面试官最爱听）

既然 Pre-Norm 训练这么稳，为什么原始论文还用 Post-Norm？因为这里存在一个‘稳定性 vs 性能上限’的经典权衡：

Pre-Norm 的代价：由于输入先被 Norm 强行拉回均值为0、方差为1，这相当于削弱了残差连接中‘原始输入 x ’的强度。模型的表示能力（Representation Capacity）会有轻微下降——通俗说，为了好训练，牺牲了一点点上限潜力。

Post-Norm 的优势：Norm 在最后，允许残差支路的原始信息尽可能多地保留和叠加，理论上模型容量的天花板更高（如果训得动的话）。

第四步：落到工程现实（终极加分项）

在实际工业界和前沿大模型中，Pre-Norm 是绝对的主流。

比如 GPT-3、PaLM、LLaMA 全系列，清一色用的 Pre-Norm（具体多是 RMSNorm，它是 LayerNorm 的简化变体）。因为在大规模训练中，训练的稳定性 >> 理论上的那点容量损失。况且，损失的上限可以通过增大模型宽度或数据量来弥补，但训练崩了（Loss 爆炸）就什么都没了。

第五步（预判追问）：如果面试官追问“那现在前沿在用什么？”

现在很多新模型（比如 LLaMA 2/3）在 Pre-Norm 基础上，把 LayerNorm 换成了 RMSNorm（去掉了均值中心化，只做缩放）。因为它计算量更小，且在 Pre-Norm 结构下，中心化带来的好处微乎其微，省下的算力可以拿去堆更多数据。

Q26：为什么是层归一化，而不是Batch归一化？

第一步：一句话定调（直接亮明观点）

在Transformer和绝大多数NLP模型中，必须使用层归一化（Layer Norm），而不能使用批归一化（Batch Norm）。这并非偏好问题，而是因为NLP数据的特性与Batch Norm的设计初衷存在根本性的冲突。

第二步：极简数学维度对比（展现基本功）

Batch Norm：在批次（Batch）维度上做归一化。它统计的是同一个Batch中，不同样本的同一位置特征的均值和方差。

Layer Norm：在特征（Feature）维度上做归一化。它统计的是同一个样本内部，跨所有特征维度的均值和方差。

（结论）：BN 依赖“样本间”的关系，LN 依赖“样本内”自身的关系。

第三步：细数 Batch Norm 在 NLP 中的“三大死穴”（核心干货）

这部分要讲出层次感，为什么 BN 在 Transformer 里完全行不通：

1. 序列长度可变（变长序列灾难）

NLP 的输入长度是不固定的（比如短句5个词，长句50个词）。BN 需要在 Batch 维度上统计同一个“位置”的统计量。

致命伤：第50个位置在短句中是 Padding（填充符），在长句中才是真实语义。如果把 Padding 和真实词混在一起算均值和方差，会引入大量噪声，导致训练崩塌。LN 只看单个样本内部的词，完美避开跨样本对齐位置的难题。

2. 训练与推理的不一致性（Online 与 Global 的冲突）

BN 在训练时用当前 Batch 的均值和方差，在推理时却要用全局滑动平均（历史所有 Batch 的统计值）。

致命伤：NLP 的分布极其灵活（例如测试集突然全是科技新闻，而训练集包含小说）。一旦推理时的全局统计量无法覆盖当前输入的特征分布，模型表现会急剧下降。LN 没有全局统计量，它永远只算当前输入自己的均值和方差，训练和推理完全一致，极其鲁棒。

3. 自回归生成时的“未来信息”泄露风险（针对 Decoder）

在 GPT 等自回归模型中，生成第 t 个词时，不能看到 $t+1$ 个词。

致命伤：如果强行用 BN，它会依赖 Batch 中其他样本的统计信息，这在分布式训练（数据并行）下，变相让当前样本窥探到了其他样本的全局分布，破坏了“样本独立性”假设。LN 每个样本独立计算，天然符合独立同分布（i.i.d）要求。

第四步：反向追问（如果你是面试官，会觉得这个候选人极其扎实）

反过来想，为什么 BN 在 CV（计算机视觉）中是王者，在 NLP 中却成了青铜？

自答：因为图像（CV）的输入尺寸通常是固定的（如 $224 \times 224 \times 3$ ），且像素值分布相对稳定，BN 可以利用 Batch 内海量的像素统计信息来加速收敛。但 NLP 的语义是离散且稀疏的，词向量维度的数值波动极大，BN 的“批间统计”根本压不住这种波动，只有 LN 的“个体内统计”才能稳住梯度的方差。

第五步：升华到现代大模型趋势（加分项，展示前沿视野）

甚至现在的 LLaMA、GPT 系列不仅用 LN，还进化成了 RMSNorm（去掉了 LN 中的均值平移，只保留缩放）。这恰恰印证了 LN 的核心思想：在 NLP 的高维空间中，特征的相对大小（缩放）比绝对数值（中心化）更重要，而 BN 那种强行拉齐批次内样本数值的做法，在语义建模中显得过于粗暴。

Q27: Encoder-only、Decoder-only 和 Encoder-Decoder 的区别吗？

第一步：一句话定调（瞬间建立框架感）

“Encoder-only、Decoder-only 和 Encoder-Decoder 的本质区别，取决于你想让模型看多宽（双向），还是只能往前看（单向），以及是否需要建立两个序列之间的映射关系。一句话总结：Encoder-only 擅长‘阅读理解’，Decoder-only 擅长‘续写生成’，Encoder-Decoder 擅长‘翻译/总结’。”

第二步：拆解核心差异（根本是“注意力掩码”）

这是最硬核的技术区分点，你一定要强调掩码（Mask）：

Encoder-only（如 BERT）：双向（Bi-directional）自注意力。没有掩码，每个词都能看到句子中所有的词（左边和右边）。这赋予了模型最强的上下文理解能力。

Decoder-only（如 GPT）：单向（Causal / Auto-regressive）自注意力。带有因果掩码（Causal Mask），每个词只能看到它左侧（过去）的词，看不到右侧（未来）的词。这是为了做自回归生成（根据前文预测下一个词）。

Encoder-Decoder（如原始 Transformer、T5）：混合架构。编码器部分是双向的（看全源文本），解码器部分是单向的（自回归生成目标文本），并且在编码器和解码器之间插入了一个交叉注意力（Cross-Attention）层，让解码器每一步都去编码器里检索最相关的信息。

Q28: Feed Forward是什么

第一步：一句话定调（极简定义）

“前馈网络（FFN）是 Transformer 中独立作用于每个位置的、简单的两层全连接神经网络。它承担了模型绝大多数的非线性变换和特征挖掘工作，是 Transformer 的‘知识存储和抽象思考’核心。”

第二步：精确的数学形式与维度变化（展现硬核基本功）

标准的 FFN 包含两个线性变换和一个非线性激活函数（原论文用 ReLU）：

升维（第一层）：输入维度先投影到高维空间

激活（非线性）：引入 ReLU（或现在的 GELU/SwiGLU），让模型具备拟合复杂函数的能力。

降维（第二层）：投影回输入维度，保持残差连接和后续层输入的维度一致。

第三步：核心哲学——FFN 和 Attention 的“分工”（面试官最爱听的高度概括）

Attention 是“词与词之间的信息交融（特征混合）”，而 FFN 是“词自身的深层思考（特征变换）”。

位置独立性（Position-wise）：FFN 在处理每个词的权重是完全共享的，且处理不同位置时互不干扰。这意味着无论句子多长，FFN 只是机械地对每个词做相同的数学运算，它不关心词的位置（位置信息全靠 Attention 那里的残差带过来）。

参数大户：在 Transformer 中，FFN 占据了模型总参数量的约 2/3（远大于 Attention 的参数量）。因此，业界常说：“大模型的知识事实和记忆，主要存储在 FFN 的权重里。”——这句话极具技术洞察力，面试官听到这里眼睛会亮。

Q29: 为什么 LoRA 微调主要调 Attention 不调 FFN?

优先只调 Attention 的性价比远高于调 FFN

1. 低秩假设的“天然适配性”（数学结构死穴）

这是最硬核的理由，面试官听到绝对内心狂喜。

Attention 的权重矩阵（ W_q, W_k, W_v, W_o ）：本质上是计算 Token 间的关联分数（相似度）。现实语义中，词与词的关系模式（如主谓宾、指代）往往是有限的、低秩的。用低秩矩阵（ $A \times B$ ）去逼近这种“关联模式”，误差极小。

FFN 的权重矩阵：FFN 是先升维（4倍）再降维，内部存储着海量的世界知识（事实、逻辑推理）。这种知识是高度复杂且非结构化的，本质上是高秩（High-rank）的。如果用极小的秩（比如 $r=8$ ）去强行逼近 FFN，相当于“用一根细线去拉一艘航母”，信息瓶颈极大，根本学不动。

2. 功能分工的“化学效应”（语义本质）

Attention：决定“在哪里找信息”（上下文路由 / 信息检索）。微调下游任务（如客服、角色扮演），主要改变的是模型“关注谁”的注意力模式。用 LoRA 微调 Attention，相当于“微调大脑的注意力开关”，极为灵敏，四两拨千斤。

FFN：决定“这些信息是什么”（知识存储 / 特征变换）。它存储的是预训练阶段积累的固化常识。如果强行微调 FFN，会过度扰动预训练的通用知识，极易引发灾难性遗忘（Catastrophic Forgetting）——模型可能学会新任务，但连“ $1+1=2$ ”这种基础事实都记不清了。

Q30: 那什么时候才会调 FFN?

虽然优先调 Attention，但在以下两种场景，调 FFN 是必要的：

持续学习（Continual Learning）注入新知识：当你要让模型学会 2026 年发生的新鲜事（比如新总统、新科技名词）时，FFN 必须被更新，因为这类事实性知识只存储在 FFN 里。

使用 MoE（混合专家）架构：如果基座模型是 Mixtral 这类 MoE 模型，FFN 本身就是‘专家路由’的核心，不调 FFN 就无法改变任务分配路由。

但即便如此，针对 FFN 的 LoRA 秩（Rank）往往要设得比 Attention 大很多（比如 Attention 用 $r=8$ ，FFN 可能要用 $r=64$ 或 128 ），才能勉强容纳信息的复杂度。”

Q30: 为什么 Q 和 V 比 K 和 O 更重要?

因为原论文实验表明, wq 控制“查询路由”, wv 控制“信息提取”, 两者搭配能最有效地改变下游任务的判别边界。而 wk (键映射) 相对稳定, wo (输出投影) 主要做维度还原, 改变它们收益极小。所以主流 Lora 默认配置都是 `target_modules=["q_proj", "v_proj"]`。

Q30: 为什么LLM只保留Decoder?

LLM只保留Decoder (即Decoder-Only架构), 并不是因为编码器 (Encoder) 没用, 而是因为在‘通用人工智能’的范式下, 编码器成了‘冗余的奢侈品’。现代LLM选择砍掉Encoder, 本质上是用‘算力’换‘通用性’, 用‘深度’换‘复杂性’的一次极致简化。

Q31: MOE模型如何实现路由激活的?

<https://aistudio.google.com/app/prompts/1Mw2FvnSP1KHKB492EY--t2fFLNCM4Fij>

注意力变体

Q1: 你为什么一开始没有直接用 MQA 或 GQA, 而是从 MHA 出发?

因为初期模型是基于通用基座 (类似 LLaMA 架构), 默认就是 MHA, 且效果稳定。我们先做 baseline, 再逐步做结构替换对比。

Q2: 那你怎么判断 MHA 成为瓶颈的?

主要是 KV cache 占用线性增长, batch 稍大就 OOM, 同时 decode 阶段带宽瓶颈明显。

Q3: 有没有量化指标?

在 8K context 下, KV cache 占用约占显存 60%+, decode latency 占总推理时间约 70%。

Q4: MQA 和 GQA 的核心区别是什么? 为什么工业界更偏 GQA?

MQA 是所有 query head 共享一组 KV heads; GQA 是分组共享, 比如 8 query heads 对 2 KV heads。

GQA 是 MHA 和 MQA 的折中方案, 能兼顾效果和推理效率。

Q5: 为什么 MQA 压缩更狠但不一定更好?

因为 KV 信息被过度共享, 会导致 attention 表达能力下降, 尤其在复杂语义对齐任务上。

Q6: 那 GQA 的“分组数”怎么选?

一般根据 Q/K head ratio 和经验设置, 比如 32Q / 8KV 或 32Q / 4KV。

Q7: GQA 在 KV cache 上具体优化了什么?

主要减少 KV cache replication, 把多个 query head 映射到共享 KV group, 从而减少 key/value 存储数量。

Q8: MLA (Multi-head Latent Attention) 你是怎么理解的?

MLA 的核心是把 KV cache 从“显式 token 维度”压缩到“latent 向量空间”, 通过低秩投影减少 KV 存储。

Q9: 它和 GQA 本质区别是什么?

GQA 是“减少 head 数”, MLA 是“压缩 KV 表示空间”。

Q10: MLA 为什么在长上下文特别有效?

因为 KV cache 是 $O(n)$ 增长, MLA 把 KV 压缩到 $O(r)$, $r \ll n$ 。

Q11: 从 MHA $\rightarrow$ GQA $\rightarrow$ MLA 的演进逻辑是什么?

核心是逐步压缩 KV 表示:

MHA: 表达最强, 最耗显存

GQA: 减少 KV heads

MLA: 压缩 KV latent space

Q12:

Q13:

二、微调

相关教程:

<https://github.com/datawhalechina/self-llm>

<https://github.com/datawhalechina/happy-llm>

LLM 微调 (☆☆)

微调体系：SFT、RLHF、DPO、ORPO、RM 训练逻辑，全参 / LoRA/QLoRA 微调适用场景；

1. 参数高效微调

LoRA

LoRA

冻结原模型，只训练低秩矩阵（A、B）去逼近权重更新。

LoRA的核心原理：用“低秩矩阵”替代“全秩更新”（给模型加一条“低秩旁路”）

QLoRA

QLoRA

(1) NF4 Quantization (4-bit量化)：这是一种基于信息论启发的全新int4量化技术。NF4量化能保障在量化过程中数据分布的一致性，换句话说，经NF4量化的权重信息损失较小，从而保证模型整体精度的最小损失。

(2) Double Quantization：对初次完成量化的常量进行二次量化，进一步缩减模型存储体积。

(3) Paged Optimizers：利用NVIDIA的统一内存管理功能，该技术可以在CPU和GPU之间自动进行页对页的传输，使得即便在GPU偶发地内存溢出（OOM）时仍能够继续进行训练。这可以理解为在训练过程中出现临时OOM时的自动故障处理机制。

(4) 增加Adapter：4-bit的NormalFloat与Double Quantization，节省了很多空间，但带来了性能损失，作者通过插入更多adapter来弥补这种性能损失。在LoRA中，一般会选择在query和value的全连接层处插入adapter。而QLoRA则在所有全连接层处都插入了adapter，增加了训练参数，弥补精度带来的性能损失，

什么是NF4?

Adapter Tuning

Prefix / Prompt Tuning

2. 全参数微调

Full Parameter Fine-tuning

核心思想：
更新全部模型参数。

特点：
效果最强（理论上）
训练成本最高
容易灾难性遗忘

3. 指令微调体系

SFT

核心：
用（instruction → response）数据训练模型。

关键点：
数据质量 > 算法
prompt格式设计很关键
多轮对话格式（ChatML / Alpaca）

多任务/混合微调

特点：
多领域混合数据训练
提升泛化能力
常用于 base model 后训练阶段

4. 对齐类微调

PPO

PPO是大模型RLHF（基于人类反馈的强化学习）的奠基性技术，它构建了一套精密而复杂的“教学评估体系”。

核心比喻：一所拥有“三堂会审”制度的精英学院。

- **学生 (Actor Model)：**我们要训练的主力模型。
- **训练数据：**只有问题 (Query)，没有标准答案。例如：“如何解释引力波？”模型必须自己独立思考并生成答案。

关键的“三位一体”评委团：

1. **Reward 模型 (RM) - “客观阅卷官”：**
 - **职责：**对模型生成的每一个答案，给出一个绝对的分数（如0-100分）。

- **特点**：必须绝对公正。它通常由另一个AI（如GPT-4）担任，或通过大量人类偏好数据训练而成。答案“北京”得高分，答案“上海”得零分。

2. Critic 模型（价值模型）- “私人心理教练”：

- **这是PPO最精妙也最难理解的部分**。既然有了阅卷官，为什么还需要教练？
- **核心问题**：分数会“欺骗”学生。做“ $1+1=2$ ”得100分，做一道复杂的微积分题可能只得60分。学生会想：“那我以后只挑简单题做就好了！”这不利于挑战难题。
- **教练的作用**：Critic模型会评估**题目本身的预期难度和学生的当前水平**。对于“ $1+1$ ”，它预期学生就该拿100分，实际拿了100分，**优势 (Advantage) 为0**，没有额外奖励。对于微积分题，它预期学生只能拿10分，结果学生拿了60分，**优势高达+50!** 这会获得巨大奖励。
- **本质**：Critic的作用是**消除题目难度差异带来的评分偏差**，鼓励模型勇于挑战并超越自我预期。

3. Reference 模型 - “初心紧箍咒”：

- 这是SFT阶段训练好的模型的一个**冻结副本**，参数不更新。
- **职责**：防止模型“耍小聪明”（Reward Hacking）。有时模型会发现，输出一堆乱码、无关信息或极端冗长的内容，反而能骗过Reward模型拿到高分。Reference模型就像一面镜子，时刻提醒学生：“看看你原来的正常样子！”如果当前生成的答案与Reference模型的输出概率分布偏离太远，就会受到惩罚，防止模型行为“跑偏”。

PPO总结：学生在“客观打分”、“因材施教”和“不忘初心”的三重监督下，每一步更新都小心翼翼（“近端”优化），稳步变强。代价是**计算成本极高**，需要同时加载和交互Actor、Critic、Reward、Reference四个模型，对显存是巨大考验。

GRPO

GRPO（组相对策略优化）—— DeepSeek的“群体赛马”哲学

核心比喻：一场内部“创意大赛”或“小组头脑风暴”。

- **关键创新**：**裁掉了“私人教练”（Critic模型）**。这一下子省去了近一半的核心显存开销，是技术上的大胆简化。
- **如何评估“优势”?**：GRPO采用了一种巧妙的“群体基线”法。
 1. **组内竞争**：对于同一个问题，让模型**一次性并行生成多个（例如4个或8个）不同的答案**。
 2. **规则评分**：针对数学、代码等有明确对错的任务，使用**规则函数**（如答案是否正确、代码能否运行通过）为每个答案打分（如对=1分，错=0分）。这取代了需要训练的Reward模型，更客观、成本更低。
 3. **计算相对优势**：计算这个小组答案的**平均分**。某个答案的得分如果高于平均分，它就是“优生”，获得正向奖励；低于平均分，则是“待改进生”，获得负向惩罚。

神奇的副作用：涌现“思维链”

这种“生多个孩子，让它们内部竞争”的机制，产生了一个意想不到的效果：模型自发地学会了“逐步推理”。

模型在试错中发现，对于复杂问题，直接蒙答案很难成为小组里的“优生”；但如果把思考步骤一步步写出来（生成思维链CoT），得出正确答案的概率和稳定性就大大提升，从而更容易获得高分奖励。

这种深度思考的能力，不是被“教”出来的，而是被GRPO的赛制环境“逼”出来的。（注：这种并行生成仅发生在训练阶段，上线推理时模型仍是单次生成，不影响速度）。

DPO / IPO / KTO

PPO和GRPO都绕不开“评分”环节（无论是RM评分还是规则评分）。DPO则提出了一个更极简、更优雅的思路：**我们能不能连“打分”都省了？**

核心比喻：一位“品味鉴赏家”的养成。

- **训练数据：**不是问题和单一答案，而是**问题 + 一对对比答案**：一个人类标注者**偏好的**答案（Chosen），和一个**不偏好的**答案（Rejected）。例如：
 - Query: “介绍你自己。”
 - Chosen: “你好，我是DeepSeek，一个乐于助人的AI助手。”
 - Rejected: “我是AI。”
- **核心目标：**DPO不要求模型绝对提高对好答案的生成概率，而是**极致地拉大“好答案”与“坏答案”在模型眼中的差距**。
- 一个反直觉的例子：
 - 训练前，模型对好答案“北京”的生成概率是99%，对坏答案“上海”的概率是1%。
 - 训练后，模型对“北京”的概率可能**降到了70%**，但对“上海”的概率**骤降到0.0001%**。
 - **这算训练失败了吗？不！这正是DPO的成功。**好坏答案之间的概率差距从98个百分点拉大到了近70个百分点，模型现在**无比确信“上海”是糟糕的选择**，即使它对“北京”的绝对信心有所波动。

DPO总结：它绕过复杂的奖励建模，**直接基于人类偏好进行优化**。训练极其稳定，显存占用小（通常只涉及一个主模型），特别擅长**精细化调整模型的风格、语气、安全性和对话品味**，但不太适合解决数学、代码等有绝对对错的任务。

RLHF

三阶段：

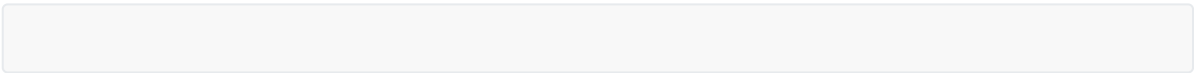
SFT（监督微调）
Reward Model（奖励模型）
PPO优化

特点：

ChatGPT经典方案
对话质量提升明显
训练复杂

5. 数据驱动增强路线

Self-Instruct / Synthetic Data



6. 长上下文专项微调

Long Context Extension / Tuning

Agent 微调

- **Agent 专有微调**：了解如何通过微调（SFT）提升模型在特定 Agent 任务上的表现（例如提升 Function Calling 的准确率、提高遵循复杂格式的能力）。
- **基于强化学习的 Agent 训练（RL for Agent）**

Embedding / Reranker 微调 (★)

Embedding微调

主流方法基于对比学习（Contrastive Learning），核心思想：

让正例对（query 与相关文档）的相似度尽量高；

让负例对（query 与不相关文档）的相似度尽量低。

常用损失函数：

Triplet Loss: anchor (query)、positive、negative 三元组，要求 $\text{sim}(q,p) > \text{sim}(q,n) + \text{margin}$ 。

InfoNCE（多分类交叉熵）：在一个 batch 内，将正例视为唯一正确类别，负例为其他所有类别，形式为 softmax 交叉熵。这是目前最主流的方式。

Multiple Negatives Ranking Loss: 适用于只有正例对、没有显式负例的场合，将同一 batch 内其他 query 的正例当作该 query 的负例。

模型评测

LLM

LLM 的评测数据集

1. **通用评测集**：[MMLU \(Massive Multitask Language Understanding\)](#)：MMLU评测模型在多种任务中的理解能力，包括各类学科和知识领域。具体包含了历史、数学、物理、生物、法律等任务类型，全面考察模型在不同学科的知识储备和语言理解能力。

- 2. **工具使用评测集**: [BFCL V2](#): 用于评测模型在复杂工具使用任务中的表现，特别是在执行多步骤操作时的正确性和效率。这些任务通常涉及与数据库交互或执行特定指令，以模拟实际工具使用场景。
- 3. **数学评测集**: [GSM8K](#): GSM8K是一个包含小学数学问题的数据集，用于测试模型的数学推理和逻辑分析能力。具体任务包括算术运算、简单方程求解、数字推理等。GSM8K中的问题虽然看似简单，但模型需要理解问题语义并进行正确的数学运算，体现了逻辑推理和语言理解的双重挑战。
[MATH](#): MATH数据集用于测试模型在更复杂的数学问题上的表现，包括代数和几何。
- 4. **推理评测集**: [ARC Challenge](#): ARC Challenge评测模型在科学推理任务中的表现，尤其是常识性和科学性问题的解答，典型应用场景包括科学考试题解答和百科问答系统的开发。[GPQA](#): 用于评测模型在零样本条件下对开放性问题的回答能力，通常应用于客服聊天机器人和知识问答系统中，帮助模型在缺乏特定领域数据的情况下给出合理的回答。[HellaSwag](#): 评测模型在复杂语境下选择最符合逻辑的答案的能力，适用于故事续写、对话生成等需要高水平理解和推理的场景。
- 5. **长文本理解评测集**: [InfiniteBench/En.MC](#): 评测模型在处理长文本阅读理解方面的能力，尤其是对科学文献的理解，适用于学术文献自动摘要、长篇报道分析等应用场景。[NIH/Multi-needle](#): 用于测试模型在多样本长文档环境中的理解和总结能力，应用于政府报告解读、企业内部长文档分析等需要处理海量信息的场景。
- 6. **多语言评测集**: [MGSM](#): 用于评估模型在不同语言下的数学问题解决能力，考察模型的多语言适应性，尤其适用于国际化环境中的数学教育和跨语言技术支持场景。

Embedding / Reranker

常见问题

三、[Agent](#)

Q1: [模型上下文限制为什么催生 Agent?](#)

3.1 核心理论 (★ ★)

3.1.1 经典架构拆解

CoT/GoT

CoT让模型:

- 不直接回答
- 而是一步一步思考

GoT让模型:

- 推理像图

ReAct

推理 + 工具调用双循环逻辑，Few-shot 构造；

如果面试官问「ReAct 本质是什么」：

ReAct 是一种 Agent 推理范式，将 LLM 的推理（Reasoning）和工具调用（Acting）交替进行。核心循环为：

Question → Thought → Action → Observation → Thought → Action → Observation → Final Answer

本质上就是一个 while 循环：

1. LLM 决定是否调用工具

2. Agent 执行工具

3. 工具结果作为 Observation 回填上下文

4. LLM 基于新的上下文继续推理

5. 直到产生 Final Answer

Plan&Solve

核心逻辑就是：**先规划 (Plan)，后执行 (Solve)**。

1. **Plan**：让大模型先制定解决问题的步骤
2. **Solve**：让大模型按照步骤执行，最终给出答案

Reflexion

核心逻辑：**大模型先生成初步答案 → 自我反思检查错误 → 基于反思修正答案**（事后的自我校正循环，一轮 Reflexion 迭代）

Self-Ask

一种提示工程（Prompt Engineering）技术，旨在让大语言模型通过**显式地提出并回答自己的中间问题**，来逐步推导出最终答案。

它的核心思想是模仿人类的思考过程：当面对复杂问题时，人们会先拆解出子问题，自言自语地逐个解答，然后综合得出最终结论。

技术	核心目标	是否规划	是否工具调用	是否反思	复杂度
CoT	提升推理能力	✗	✗	✗	★
GoT	提升复杂推理	部分	✗	✗	★★
Self-Ask	拆解问题	部分	可选	✗	★★

技术	核心目标	是否规划	是否工具调用	是否反思	复杂度
Plan & Solve	先规划后执行	✓	可选	✗	★★★
ReAct	推理+行动	动态	✓	✗	★★★★
Reflexion	自我纠错	动态	✓	✓	★★★★★

3.1.2 三大核心模块

- **工具调用 (Function Calling)**：函数描述构造、参数校验、错误重试、多工具串联；
- **记忆系统 Memory**：短时上下文记忆、向量库长期记忆 (RAG+Agent 结合)、分层记忆；
- **规划 Planning**：任务拆解 (子任务拆分)、动态纠错、多轮迭代重规划。

3.1.3 Agent 常见痛点

一、幻觉 (Hallucination)

表现：Agent 凭空捏造事实、虚构不存在的工具参数，或在回答中给出错误的信息。

- **优化方案**：
 1. **RAG (检索增强生成) 深度融合**：
 - 在 Agent 决策前或生成阶段，接入知识库 (向量数据库、图数据库或企业内部搜索接口)，将确定性的事实作为 Context (上下文) 喂给大模型，强制大模型“基于参考资料回答，若无参考则承认不知道”。
 2. **自我审视与纠错机制 (Self-Reflection)**：
 - 引入自我反思 workflow (如 Reflexion 框架)。在 Agent 产出最终答案或执行动作前，设计一个内部提示词 (或引入一个更轻量/更专注的 Critic Agent) 对输出进行“事实性校验” (Fact-checking)，若发现不合逻辑或无事实依据，则进行内部重打回。
 3. **约束性解码 (Constrained Decoding)**：
 - 利用特定框架 (如 Outlines、Instructor 或 JSON Mode) 强制 Agent 的输出必须符合特定的 Schema (如严格的 JSON 格式)。这能极大降低大模型在输出格式和参数命名上的“幻觉”。

二、工具错选 (Incorrect Tool Selection)

表现：在面临多个工具时，Agent 选择了错误的工具，或无法正确理解工具的参数要求。

- **优化方案**：
 1. **分层路由与工具动态注入 (Hierarchical Routing)**：
 - 避免一次性把几十个工具一股脑推给 Agent (这会干扰大模型的注意力)。采用**两阶段路由**：先由一个高层 Agent (Router) 决定当前任务属于哪个大类，再将该大类下的少数几个具体工具注入给下层 Agent，减少干扰项。
 2. **优化工具的语义描述 (Semantic Description)**：

- 工具的 name (名称) 和 description (描述) 是大模型选择工具的唯一依据。应像写高质量 Prompt 一样设计工具描述：清晰说明工具的“适用场景”、“输入参数格式”及“输出结果代表什么”。

3. 少样本学习 (Few-Shot Examples) :

- 在 System Prompt 中加入具体的 Few-shot 示例，展示在面临模糊或相似请求时，如何正确甄别并选择相应的工具。

4. Schema 强校验与反馈环:

- 当 Agent 传错工具参数时，代码层 (如 Pydantic 校验) 应捕获错误，并将具体的报错信息 (如“参数 age 应该为整型，但你输入了字符串 'twenty'”) 作为反向 Observation 反馈给大模型，引导其自动修正并重新调用。

三、循环调用 (Infinite Loop / Repetitive Calling)

表现: Agent 陷入死循环，不断用相同的参数调用同一个工具，或者在某两三个步骤之间反复打转，无法推进任务。

• 优化方案:

1. 硬性截断 (Max Iterations & Timeouts) :

- 在 Agent 执行引擎 (Executor) 中设置硬性上限，如最大迭代步骤 (max_iterations=5) 或最大执行时间限制。达到上限后强行中断，并向用户报错或转入人工客服。

2. 动作历史去重与相似度检测 (Action History De-duplication) :

- 维护一个执行历史队列 (History Trace)，记录每次的“Tool-Input-Output”。在每次决策前，通过比对新生成的 Action 与历史 Action 的相似度 (或完全匹配)，一旦检测到连续 2 次及以上完全一致的调用，即判定为陷入死循环，触发“破局提示” (Breaker Prompt) 或中断。

3. 引入显式状态机约束 (State Machine / Graph-based) :

- 放弃纯粹由大模型自主控制的 ReAct 自由流，改用基于状态图的框架 (如 LangGraph)。将复杂的任务拆解为节点 (Nodes) 和有向边 (Edges)，限制节点之间的跳转逻辑，从架构层面阻断非法循环。

四、任务越界 (Task Boundary Violation)

表现: Agent 受到用户提示词的诱导 (Prompt Injection)，开始执行与自身定位不符的任务，或越权访问敏感资源。

• 优化方案:

1. 安全护栏机制 (Guardrails) :

- 在输入和输出端部署防护层 (如 Guardrails AI、LlamaGuard 或自定义敏感词/分类模型)。
- **输入护栏:** 检测用户请求是否包含恶意注入、越权尝试或偏离主题的提问，提前拦截。
- **输出护栏:** 在 Agent 生成内容后、交付给用户前，检测是否包含敏感数据泄露或违规言论。

2. 最小特权原则与环境隔离 (Least Privilege) :

- 不要给予 Agent 对应的 API Key 或数据库连接过高的权限。在基础设施层实现 RBAC（基于角色的权限控制），Agent 只能调用其职责范围内的受控 API。

3. 关键步骤引入人机协同 (Human-in-the-Loop, HITL)：

- 对于高风险、破坏性、涉及资金或敏感信息修改的操作（如发送邮件、转账、删除数据），设计硬性的人工审批节点，只有人类确认后，Agent 才能继续执行。

五、多步骤路径跑偏 (Multi-step Path Deviation)

表现：在长链条 (Long-horizon) 任务中，Agent 随着执行步骤的增加，逐渐忘记了最初的终极目标，或者由于某一步工具返回了干扰信息，导致后续的执行路径完全漂移。

• 优化方案：

1. “计划与执行”解耦 (Plan-and-Solve)：

- 不要让 Agent 边想边做（传统的 ReAct 容易迷失）。采用“先规划，后执行”的模式。
- **Planner**：先生成一个完整的多步骤执行计划 (Step 1, Step 2, Step 3...)。
- **Executor**：逐一执行计划中的子任务，并在每个子任务完成后更新状态。

2. 动态重规划与目标锚定 (Dynamic Re-planning)：

- 在每次工具执行完毕后，要求 Agent 评估：1. 原始目标是什么？ 2. 当前进度到哪里了？ 3. 刚才的工具结果对既定目标是否有帮助？ 4. 是否需要修正后续的计划？ 将全局目标 (Original Goal) 始终固化在 Context 的醒目位置（目标锚定）。

3. 记忆管理与上下文压缩 (Memory Summary)：

- 长链条任务会导致 Context Window 极速膨胀，冗余的中间工具输出会稀释大模型对核心目标的注意力。
- 不建议直接把所有的历史对话和工具输出塞给模型。应当采用**滑动窗口记忆**或**摘要记忆 (Summary Memory)**，只保留“已达成的阶段性成果”和“接下来的行动点”，过滤掉无用的调试日志和工具冗余返回。

3.2 Agent框架

LangGraph

很多企业已经从LangChain迁移到LangGraph。

需要掌握：

- StateGraph
- Node
- Edge
- Conditional Edge
- Memory
- Checkpoint
- Human-in-the-loop

最好能手写：

- ReAct Agent
- Plan-and-Execute Agent
- Multi-Agent

OpenClaw

常见问题

Q1：AutoGPT、BabyAGI、OpenAI Agent 框架设计思路

AutoGPT解决的是“让模型自主行动”，BabyAGI解决的是“让模型管理任务”，而OpenAI Agent解决的是“让模型可靠地调用工具并完成业务流程”。

3.3 Tool Calling

Function Calling = 早期官方叫法

Tool Calling = 现在的通用、升级版叫法

Agent本质：LLM + Tool + Memory + Planning

很多人在学习 Agent 时，会把 **Tool Calling** 和 **Function Calling** 当成同一个概念，但实际上它们是 **包含关系**：

Function Calling 是一种实现机制
Tool Calling 是一种能力范式



MCP

“MCP（Model Context Protocol）是由 Anthropic 最近推出的一个**开放标准协议**，它的核心目的是**解决 AI 模型与外部数据源、工具之间的连接壁垒问题**。

MCP协议理解为，使用 mcp.server.fastmcp.FastMCP 对象，将python函数转换为大模型能理解的固定格式描述

可以将 MCP 协议中的 FastMCP 对象理解为一个**‘翻译器’和‘适配器’**：它自动将 Python 函数的函数名、文档字符串（Docstring）、参数类型注解（Type Hints）进行解析，封装成大模型能够通过 JSON 格式高效处理的 **Schema（结构定义）**。

重点：

- MCP Client（MCP 客户端）
- MCP Server（MCP 服务端）
- Tool Discovery（工具发现机制）
- Tool Invocation（工具调用流程）

很多公司已经开始要求：

会开发MCP Server

Skill

主流出现在 **Agent、Function Calling / Tool Calling** 体系里：

- 本质：**大模型可调用的「可执行能力单元」**
- 形态：函数、接口、脚本、插件，带固定入参、出参、描述
- 核心流程：LLM 理解用户意图 → 选择 Skill → 拼装参数 → 调用执行 → 拿回结果继续对话
- 特点：**绑定单个 Agent / 应用**，能力封闭，多用于单智能体内部工具调用

维度	Skill / Tool(Function Calling)	MCP (Model Context Protocol)
定位	功能单元、能力集合	通信协议、交互标准
层级	应用层「功能」	传输层 / 会话层「协议规范」
耦合度	强耦合：Skill 写死在 Agent 代码 / 配置里	完全解耦：客户端和服务端分离，跨进程 / 跨机器
作用范围	单 Agent 内部使用	通用互通：任意 LLM 客户端 ↔ 任意 MCP 服务
能力范围	侧重 工具函数调用	不止工具，还包含 上下文读写、资源访问、会话、文件、日志等
生态目标	给单个智能体装插件	打造统一的 AI 资源互联生态

Function Calling / Tool Calling

让 LLM 不直接回答问题，而是生成一个函数调用请求。

需要熟悉

- Function Calling（目的是调用工具）
- JSON Schema（告诉大模型函数参数长什么样）
- Structured Output（目的是结构化结果）
- Tool Routing（从多个工具中选择最合适的工具）

高频问题

Q1: Tool Description设计

- （1）说清楚什么时候用
- （2）说清楚什么时候不用
- （3）参数解释清楚
- （4）提供样例

Tool Description本质是在给模型做路由指导。我会重点描述工具适用场景、不适用场景、参数含义以及调用示例，因为**Tool Calling**错误很多时候不是模型能力问题，而是**Description**歧义导致的。

Q2: Few-shot

如何利用示例提升模型表现？

生产环境一般：3~10个样例

重点覆盖：高频场景；易混淆场景；边界场景

Few-shot本质是给模型建立输入输出映射关系。在分类、工具选择和结构化输出任务中效果非常明显。我通常会优先覆盖容易混淆的**Case**，而不是一味增加样本数量。

Q3: Tool选择机制

Agent如何决定调用哪个**Tool**？

方案1: 纯LLM **Routing**（简单，但不稳定）

方案2: **Rule** + LLM（Query → 规则过滤 → 候选**Tool** → LLM选择）

方案3: **Embedding Routing**（工具向量化，Query Embedding → Tool Embedding → Similarity Search → TopK Tool → LLM选择）

Tool选择一般不完全依赖LLM。我更倾向于**Rule Filter** + **Embedding Recall** + LLM **Decision**的三层路由架构，先缩小候选范围，再让模型决策，可以显著降低误调用率和**Token**成本。

Q4：如何提高Tool Calling准确率？

方法1：优化Tool Description
方法2：增加Few-shot
方法3：减少候选Tool数量
方法4：参数校验
方法5：Tool Retry
方法6：Self-Reflection
方法7：分层Agent

回答

提高Tool Calling准确率我通常从五个方面优化：

第一，设计清晰的Tool Description，明确适用和不适用场景；

第二，通过Few-shot给模型提供正确调用示例；

第三，采用Rule Filter或Embedding Recall缩小候选Tool范围，而不是让模型直接从大量工具中选择；

第四，使用JSON Schema或Pydantic进行参数约束和校验；

第五，引入Retry和Reflection机制，让模型能够根据错误信息重新选择或修正参数。

在实际Agent项目中，Tool Calling问题大约70%来自工具描述和路由设计，而不是模型本身，因此我会优先优化Tool设计和路由层。

Q5：工具调用数据集构造

Q6：如何优化模型原生 Function Calling 能力

通常希望考察你是否理解 Prompt → Schema → Tool Selection → Parameter Extraction → Tool Execution → Result Injection 整条链路。

1. Tool Description 优化（收益最大）
2. Few-shot Function Calling
3. JSON Schema 优化
4. Tool Routing（工具路由）
5. ReAct / Multi-step Function Calling（对复杂任务采用 ReAct 或 Multi-Step Agent，使模型具备多轮工具调用能力，而不是单轮 Function Calling。）
6. Function Calling 数据微调（最高阶）

回答

我通常从六个层面优化：

Tool Description 优化，明确工具职责、边界和调用示例；

Few-shot Tool Calling，通过示例教会模型何时调用工具以及如何填充参数；

JSON Schema 优化，增加字段描述、枚举约束和参数校验规则；

Tool Routing，先召回 Top-K 候选工具，减少工具干扰；

Multi-step Agent，利用 ReAct 支持复杂任务的多轮工具调用；

基于 Tool Call Logs 的 SFT 微调，从根本提升工具选择和参数抽取能力。

在实际项目中，收益通常遵循：

Tool Description > Tool Routing > Few-shot > JSON Schema > SFT

前四项往往无需训练即可显著提升 Function Calling 准确率，是生产环境中最常见的优化手段。

3.4 RAG (★)

3.4.1 文档解析

- **痛点：** 如何解析复杂的 PDF、Word、PPT 格式？特别是双栏排版、表格 (Table)、图片、OCR 识别。
- **核心技术：** 布局分析 (Layout Analysis, 如 LayoutLM、PaddleOCR)、专门解析表格的工具 (如 Table Transformer、Unstructured、PyMuPDF) 。

3.4.2 Chunk策略

Fixed Chunk (固定大小分块)

工作原理：

固定大小分块是最简单、最基础的分块方法。它直接按照固定的字符数 (Character-based) 或 Token 数 (Token-based) 将文本切分成大小相同的块。为了避免语义在边界处被硬生生切断，通常会设置一个重叠区 (overlap)。

例如：chunk 大小设为 500 字，重叠区设为 50 字。这意味着第二个 chunk 会包含第一个 chunk 的最后 50 个字。

优点：

- 实现非常简单，计算开销极低。
- 容易预测和控制 Token 的消耗。

缺点：

容易破坏语义完整性。即使有重叠区，一句话或一个完整的段落也可能被切成两半，导致检索时丢失上下文。

适用场景：

- 格式非常规整、没有复杂段落逻辑的文本。
- 原型开发阶段，或对检索精度要求不高的快速基线测试。

Semantic Chunk (语义分块)

工作原理：

语义分块不再依赖硬性的字数限制，而是根据文本内容的意思来进行切分。通常的做法是：

1. 先将文本切成句子。
2. 计算相邻句子之间的向量相似度 (Embedding Similarity)。
3. 如果相邻两句的相似度低于设定阈值，说明话题发生了转换，就在这里进行切分。

此外，简单一些的语义分块也可以基于 Markdown 标题、段落分隔符 (\n\n) 或标点符号 (如句号) 来进行。

优点：

- 能最大程度保证每个 chunk 内部语义的完整性和独立性。
- 检索出来的文本块通常是一个完整的观点或主题，有利于 LLM 理解。

缺点：

计算成本较高。如果使用向量相似度切分，需要对每个句子生成 **Embedding** 并计算相似度。
Chunk 的大小不固定，可能会出现极长或极短的 **Chunk**，较难控制输入给 **LLM** 的 **Token** 数量。
适用场景：
叙事性较强、段落主题明确的文档（如小说、法律条文解释、分析报告）。

Parent Child Chunk (父子分块 / 检索与生成分离)

工作原理：
父子分块的核心思想是**“小块检索，大块输入”**。它将文档切分为两种尺寸的块：
子块（**Child Chunk**）： 较小的文本块（例如 **100-200 Token**），用于生成向量并进行索引。
父块（**Parent Chunk**）： 较大的文本块（例如 **1000-2000 Token**，或者子块所在的整个段落/章节）。
在检索阶段，系统在向量数据库中匹配最相似的子块。但是，在将检索结果喂给 **LLM** 进行生成时，系统会根据子块的 **ID** 自动替换并返回其对应的父块。

优点：
解决了检索和生成的矛盾。小块在向量化时语义更聚焦，检索更精准；大块在生成时能提供丰富的上下文，防止 **LLM** 产生幻觉或断章取义。
缺点：
数据结构较为复杂，需要在数据库中维护父子关系的映射。
存储开销略微增加。
适用场景：
专业技术文档、**API** 文档、学术论文。这些文档中往往包含细节数据（适合子块定位），但回答问题需要结合上下文（适合父块生成）。

Hierarchical Chunk (层级分块)

工作原理：
层级分块可以看作是父子分块的延伸或更复杂的变体。它将文档组织成一个多级的树状结构（例如：文档 -> 章节 -> 段落 -> 句子，或者通过算法将相似的 **Chunk** 进行聚类并生成高层摘要）。
一个典型的实现是 **RAPTOR** 算法：它先将底层叶子节点（细粒度文本）进行聚类，然后用 **LLM** 为每个聚类生成一个摘要（高层节点），再对这些摘要进行聚类生成更高层的摘要，最终构建出一棵树。
在检索时，系统会根据问题的范围（大局观问题还是细节问题），决定从树的哪一层进行检索。

优点：
既能回答微观/细节问题（如“某某产品的尺寸是多少？”——定位到叶子节点），也能回答宏观/全局问题（如“这本书的核心观点是什么？”——定位到高层摘要节点）。
缺点：
实现和维护成本极高。
如果使用 **LLM** 逐层生成摘要，建库阶段（**Embedding** 和 **Prompt** 消耗）的成本会非常高昂。
适用场景：
长篇研究报告、整本书籍分析、复杂的合规性文档等需要兼顾宏观与微观视角的信息提取任务。

3.4.3 检索增强

查询变换与扩展

常见方法：

1. Query Rewrite (查询改写)
2. Query Expansion (查询扩展)
3. Multi Query

Sparse Retrieval

稀疏检索,传统搜索引擎使用的方法

常见方法：

1. TF-IDF
2. BM25

优点

快,可解释

缺点

无法理解语义

Dense Retrieval

用Embedding做检索

常见Embedding模型

OpenAI: text-embedding-3-large

BGE: bge-large-zh、bge-m3

Qwen: Qwen3-Embedding

优点

理解语义

缺点

精确匹配较差

Graph Retrieval

适用于

知识图谱

代码仓库

Agent Memory

企业知识库

常见项目

GraphRAG

Hybrid Search

实际生产最常用

Sparse Retrieval + Dense Retrieval

主流实现

Elasticsearch: BM25 + Vector

Pinecone: Hybrid Search

Weaviate: Hybrid Search

Milvus: BM25 + Dense

Rerank

有时 Rerank 比 Retriever 更重要

常见结构

Cross Encoder

常见Rerank模型

BGE: bge-reranker-large

Cohere Rerank

Qwen: Qwen3-Reranker

3.4.4 向量数据库

- [Milvus](#)
- [Qdrant](#)
- [Weaviate](#)

高频问题

Q1: Cross Encoder vs Bi Encoder

掌握 Agentic RAG (智能体检索) : Agent 自主决定何时检索、检索什么、如何重构 Query、如何评估检索结果 (Self-RAG, Corrective RAG) 。

3.5 Context Engineering (☆☆)

上下文工程 (Context Engineering) 这是2026年越来越火的方向。

核心思想: 如何在有限上下文窗口内, 为模型动态构造“最有价值的输入环境 (Context) ”。

3.5.1 Prompt Engineering

提示词工程, 就是设计、优化、调试给大语言模型的输入指令 (Prompt / 提示词) , 在不改动模型本身的前提下, 让 AI 精准按照你的要求输出结果的技术与方法。

自动提示词优化

自动提示词优化 (Automated Prompt Optimization, APO)

1. 为什么需要 APO?

在日常开发中, 传统的提示词工程通常由人类工程师手动完成:

1. **耗时且效率低**: 改一个词, 需要重新测试成百上千条数据。
2. **容易“拆东墙补西墙”**: 你为了解决 Case A 修改了提示词, 结果却导致原本正确的 Case B 和 Case C 报错了。
3. **不可迁移**: 当你把底层大模型从 GPT-4 换成 Claude 时, 原本精雕细琢的提示词可能直接失效, 所有工作又得重来。

APO 的出现，就是为了把这种“手工业”变成类似于传统机器学习的“工业化训练”。

2. APO 是如何工作的？

APO 的核心思想是**将机器学习中的“模型训练（梯度下降）”逻辑套用在文本上**。它将提示词视为“待训练的参数”，通过以下循环自动迭：

1. **准备数据集 (Dataset)**：包含输入数据（如“请翻译这句话”）和标准答案（Ground Truth）。
2. **运行与评估 (Loss Function)**：使用初始提示词在数据集上运行，用算法（或另一个 LLM 裁判）评估输出质量，找出出错的案例（Bad Cases）[\[1\]](#)。
3. **错误分析 (Gradient/梯度)**：让一个“评判大模型”分析这些出错的案例，生成自然语言形式的诊断报告（例如：“当前提示词没有明确规定输出格式，导致大模型输出了多余解释”）。这在 APO 中被称为**“文本梯度” (Textual Gradient)**。
4. **生成候选提示词 (Optimizer)**：让一个“优化大模型”根据诊断报告，修改并生成几个新的候选提示词。
5. **筛选与迭代 (Selection)**：在验证集上测试这些新的候选提示词，保留表现最好的一个（或使用束搜索 Beam Search 等算法保留前几名），然后进入下一轮迭代，直到性能达到瓶颈。

3. 代表性的 APO 技术

在学术界和工业界，目前有几种主流的 APO 实现方案：

- **APO / [PromptWizard](#) (微软)**：提出使用自然语言“梯度”来批评当前的提示词，并在语义相反的方向上自动重写提示词。
- **OPRO (Google DeepMind)**：将大模型作为优化器 (LLMs as Optimizers)。它在 prompt 中展示历史尝试过的提示词及其对应的得分，让大模型根据这些“历史轨迹”自动推导出得分更高的新提示词。
- **DSPy (斯坦福)**：将提示词工程转变为“声明式编程”。你只需要定义输入输出，DSPy 的编译器 (Compiler) 会自动帮你优化提示词、自动挑选最合适的 Few-shot 示例，完全避开手动写 Prompt 的过程。
- **TextGrad (斯坦福)**：将上述过程进一步泛化。它不仅能优化 Prompt，还能自动优化代码和文本答案，把整个 LLM 系统的优化做得像 PyTorch 的自动求导一样系统化。

4. APO 的优势

- **数据驱动**：提示词的优化结果完全基于真实测试集的反馈，避免了人类的主观偏好。
- **抗回归 (Regression-proof)**：在迭代过程中，系统会确保新的提示词在整个数据集上的综合表现变好，而不会因为解决了一个新 Bug 而引入三个旧 Bug。
- **自动化适配大模型**：当企业更换底层模型时，只需把新模型接入 APO 管道，重新跑一遍自动化优化流程，就能在几小时内得到一套最契合新模型的提示词。

高频问题

Q1: prompt优化

(1) 角色定义 (Role): 明确模型身份

(2) 任务拆解 (Task Decomposition): 不要让模型一次做很多事。(Chain of Thought、Structured Prompting)

(3) 输出约束 (Output Constraint): 降低格式错误率, 提高自动化处理能力

(4) 边界条件说明: 比如 不知道时输出: UNKNOWN, 不要编造信息

3.5.2 Context Selection

核心问题:

哪些信息应该放进上下文?

常见方法:

Rule-Based

保留最近10轮

Embedding Retrieval

query_embedding → TopK Similar Context

LLM Selection

让LLM决定保留哪些内容

3.5.3 Context Compression

常见方法:

Summarization

100轮对话 → 摘要

Semantic Compression

保留结论
删除冗余细节

Structured Compression

主要解决大语言模型（LLM）在处理**长上下文**、**多轮对话**时的三个痛点：

- **超出上下文窗口**：几轮对话后Token过多导致模型“忘记”开头。结构化压缩后，长对话被精简为几十个Token的关键字段。
- **注意力分散**：原始对话中充斥“嗯...那个...刚才你说的.....”等噪声。提取JSON结构能引导模型直接聚焦于 `intent` 和 `constraint`，提升任务成功率。
- **成本与延迟**：压缩后的JSON输入比原始长文本短得多，能显著降低API调用成本和响应延迟。

```
{  
  "intent": "...",  
  "constraint": "...",  
  "history": "..."  
}
```

原始输入（啰嗦，约80 Token）：

用户A：“嗨，你好，我想了解一下，就是.....呃.....我下周可能要去上海，你知道的，上海天气怎么样？哦对了，顺便问一下，去那边需不需要带伞啊？大概下周三的样子。”

助手：“好的，请问是下周三吗？”

用户A：“对对对，就是下周三。”

结构化压缩后（仅约20 Token）：

```
{  
  "intent": "查询天气",  
  "constraint": {"地点": "上海", "时间": "下周三", "关注点": ["是否需要带伞"]},  
  "history": "用户确认时间为下周三"  
}
```

典型框架：

- MemGPT
- LangGraph Memory
- Claude Memory

3.5.4 Retrieval Engineering

RAG实际上属于Context Engineering的重要组成部分。

常见方法：

Query Rewrite

用户：

它什么时候发布的？

改写：

GPT-5什么时候发布？

Retrieval

BM25
Dense Retrieval
Hybrid Search
Graph Retrieval

Rerank

Top50 → Top5

Context Packing

把检索结果组织进Prompt。

3.5.5 Memory Engineering

为何智能体需要记忆与RAG？

1) 局限一：无状态导致的对话遗忘

当前的大语言模型虽然强大，但设计上是无状态的。这意味着，每一次用户请求（或API调用）都是一次独立的、无关联的计算。模型本身不会自动“记住”上一次对话的内容。这带来了几个问题：

上下文丢失：在长对话中，早期的重要信息可能会因为上下文窗口限制而丢失

个性化缺失：Agent无法记住用户的偏好、习惯或特定需求

学习能力受限：无法从过往的成功或失败经验中学习改进

一致性问题：在多轮对话中可能出现前后矛盾的回答

2) 局限二：模型内置知识的局限性

除了遗忘对话历史，LLM 的另一个核心局限在于其知识是静态的、有限的。这些知识完全来自于它的训练数据，并因此带来一系列问题：

知识时效性：大模型的训练数据有时间截止点，无法获取最新信息

专业领域知识：通用模型在特定领域的深度知识可能不足

事实准确性：通过检索验证，减少模型的幻觉问题

可解释性：提供信息来源，增强回答的可信度

为了克服这一局限，RAG技术应运而生。它的核心思想是在模型生成回答之前，先从一个外部知识库（如文档、数据库、API）中检索出最相关的信息，并将这些信息作为上下文一同提供给模型。

Short-term Memory

- **工作记忆 / 短期记忆**
- **定义：**Agent 当前正在进行的任务链条中所必需的即时信息（例如正在运行的代码片段、当前的子目标）。
- **技术实现：**通常直接驻留在 LLM 当前的上下文窗口中，生命周期较短，任务结束即释放。

Long-term Memory

长期记忆, 将信息持久化到外部存储系统。

Conversational Memory

- **对话记忆**
- **定义：**单次或跨会话（Session）中，用户与 Agent 之间的聊天历史记录。
- **技术实现：**SQL 关系表、NoSQL 键值对，通过滑动窗口（Sliding Window）或自动摘要（Summary）在输入时动态压缩和加载。

Episodic Memory

- **情景 / 事件记忆**
- **定义：**记录“过去发生了什么”。它保存了 Agent 的历史行为轨迹、调用过的工具日志（Tool Logs）、执行成败以及从中获得的反馈（即“前车之鉴”）。
- **技术实现：**利用事件溯源（Event Sourcing）设计，通过时间戳、成功/失败标签将每一次的执行轨迹序列化保存，支持相似任务场景下的少样本检索（Few-shot Retrieval）。

Semantic Memory

- **语义记忆**
- **定义：**Agent 掌握的静态或相对持久的事实、概念与常识（即 RAG 所依赖的外部知识库）。
- **技术实现：**向量数据库、混合检索系统（Hybrid Search）。

Procedural Memory

- **程序记忆 / 工作流与工具记忆**
- **定义：**关于“如何做一件事”的记忆。包括 Agent 需要遵循的具体操作规程、业务流，以及它所拥有的工具箱定义（Toolbox）。
- **技术实现：**动态工具检索系统。当 Agent 拥有成百上千个 API 时，不应把所有工具描述直接丢进 Prompt，而是将工具定义存储在数据库中，由 Agent 动态语义检索并加载当前需要的工具。

Entity Memory

- **实体 / 结构化记忆**
- **结构化定义**：关于特定人、项目、系统或对象的结构化事实（例如：用户 A 喜欢用 Python、项目 B 的截止日期是明天）。
- **技术实现**：通常使用图数据库（Knowledge Graph）或带有 Vector 扩展的关系型 SQL 表，用于维护高度结构化的属性关联。

3.5.6 Tool Context Engineering

工具上下文工程，Agent大量依赖Tool。

核心问题：

Tool结果如何注入上下文？

Tool Description

Tool Output Formatting

Tool Routing

Function Calling Schema

3.5.7 Agent Planning Context

规划上下文，Agent决策依赖上下文。

ReAct

Plan-and-Solve

Reflexion

Tree of Thoughts

3.5.8 Context Optimization

（上下文优化）

Token Budget Allocation

Context Ranking

Dynamic Context

面试可能直接问：

如何解决200K上下文导致的性能下降？

3.6 Harness Engineering (☆☆)

3.7 能力评测 (☆☆)

这是很多候选人的短板。

重点：

- **评测方法**：如何评估一个 Agent 的好坏？了解自动评测（如 Ragas、G-Eval、使用 GPT-4 作为裁判）与人工评测的结合。
- **链路追踪**：熟练使用或了解 Agent 监控工具（如 LangSmith、Langfuse、Phoenix），能够分析 Agent 运行过程中的瓶颈（如哪一步调用耗时最长、哪一步 Prompt 出现幻觉）。
- **评估**：如何判断Agent任务成功？设计成功率、步骤效率、工具调用准确率等指标。熟悉 LangSmith或自建简单的trace log。
- **可观测性**：记录每一步的thought/action/observation，方便调试。面试中展示你做的Agent能看到完整执行轨迹。
- **评估体系**：了解 AgentBench、GAIA 等主流评测集。
- **Trace 分析**：如何追踪 Agent 的思考链路（Thought Trace），定位是推理错了、检索错了还是工具调错了。
- **LLM-as-a-Judge**：使用更强的模型来自动评估 Agent 任务完成度的方法论。

3.7.1 RAG能力评测

RAG系统的评估通常分为两个核心部分：**检索阶段 (Retrieval)** 和**生成阶段 (Generation)**。

1. 评估方法

行业内目前最广为接受的评估方法是**RAG三元组 (RAG Triad) 框架**（由TruEra/TruLens率先提出）[\[3\]](#)，它将评估细化为三个维度：

- **上下文相关性 (Context Relevance)**：检索出来的文档片段是否真的与用户的提问相关？（评估检索器，避免引入无关噪声）[\[2\]](#)

- **忠实度/幻觉度 (Faithfulness / Groundedness)**：大模型生成的回答是否完全基于检索出来的上下文？有没有凭空捏造事实？（评估生成器，检测幻觉）
- **回答相关性 (Answer Relevancy)**：大模型最终的回答是否真正解答了用户的提问？[2]

此外，还有以下传统和辅助评估指标：

- **传统检索指标**：命中率 (Hit Rate)、平均逆文档排名 (MRR)、NDCG、召回率 (Recall) 等（用于独立调优向量数据库和检索算法）[1]。
- **LLM-as-a-Judge (大模型作为裁判)**：使用能力更强的大模型（如 GPT-4）对生成质量进行语义层面的打分或成对比较 (Pairwise Comparison)。

2. 主流评测工具

- **Ragas (Retrieval Augmented Generation Assessment)[2]**
 - **特点**：目前最流行的开源RAG评测框架。基于研究论文提出了一系列无参考 (Reference-free) 指标，无需人工标注的黄金标准 (Golden Dataset) 即可利用大模型对上下文精度、忠实度、回答相关性进行自动化打分[2]。
- **DeepEval (Confident AI)[2]**
 - **特点**：开源的LLM系统单元测试框架，完美兼容 Pytest[2]。它提供了丰富的RAG评估指标，并且由于其单元测试的属性，极易与CI/CD管道集成，适合在代码部署前进行回归测试。
- **TruLens (Truera)[3]**
 - **特点**：RAG三元组概念的鼻祖[3]。提供强大的可视化面板，能够直观地追踪哪一次查询的哪个环节（检索还是生成）出现了瓶颈，非常适合开发阶段的调试和调优。
- **Arize Phoenix**
 - **特点**：开源的LLM可观测性和评估平台[2]。支持OpenTelemetry协议 (OTel-native)，能够将检索到的文本转化为向量空间进行可视化 (UMAP)，方便定位检索“死角”[3]。
- **LangSmith / Langfuse**
 - **特点**：链路追踪 (Tracing) 领域的顶流工具。虽然它们本身是可观测性平台，但提供了强大的数据集管理、人工反馈收集和自动打分流程，对串联了多步检索的复杂RAG管道十分友好。

3.7.2 Agent能力评测

相比RAG，Agent（智能体）的评估要复杂得多。Agent 通常涉及多轮对话、工具调用 (Tool Use)、自我规划 (Planning) 以及外部环境交互。它的失败往往是组合性的（例如：即使每一步工具调用都正确，但整体逻辑推理错误，导致任务失败）[9]。

1. 评估方法

- **轨迹评估 (Trajectory & Trace Evaluation)**：不仅看最终结果，更要分析Agent决策路径中的每一步[7]。例如：是否调用了正确的工具？传参是否正确？是否存在死循环或重复无意义尝试？
- **任务成功率 (Task Success Rate)**：端到端的通过率测试 (Pass/Fail)。判断Agent在沙盒环境（如代码库、模拟网页）中操作后，是否达到了最终预期状态。
- **效率与成本指标**：
 - **执行步数 (Steps)**：达到目标用了多少步？
 - **消耗Token及费用 (Cost)**：单次任务消耗了多少API费用[11]？

- **时间 (Latency)**：执行任务耗时多久？
- **异常处理与纠错能力 (Robustness)**：当遇到工具调用报错、网络超时或用户干扰时，Agent是否能够自我纠错并继续执行。

2. 评测基准 (Benchmarks)

在Agent评测中，高质量的测试集/模拟环境（基准）比单纯的工具更关键。目前业界公认的主流基准有：

- **SWE-bench**：软件工程智能体基准[8]。要求Agent在真实的、庞大的GitHub开源代码库中定位并解决真实的Issue[8]。由于需要读写代码、运行测试，这是目前难度最高、最贴近生产环境的评测之一。
- **GAIA (General AI Assistants)**：通用AI助手基准[8]。包含各种需要多模态理解、长跨度推理、复杂工具使用（如查网页、读取PDF、运行Python脚本等）的实际生活与工作任务[8]。
- **BFCL (Berkeley Function-Calling Leaderboard)**：加州大学伯克利分校提出的函数调用基准[8]。专门用来评估模型在单步/多步/并行工具调用、参数提取上的准确率，是智能体工具调用能力的“黄金标准”。

3. 评测与可观测性工具

由于Agent链路的非确定性，评测工具往往与可观测性追踪（Tracing）深度绑定：

- **LangSmith (针对 LangGraph/LangChain 等)**
 - **特点**：在评估多轮对话、多Agent协作（Multi-agent）和状态机工作流方面极具优势。它能清晰展现每个Agent的决策分支、记忆读取和工具交互 trace。
- **DeepEval (Confident AI)**
 - **特点**：引入了针对Agent的专项评估指标（如Tool Correctness），支持对带有有向无环图（DAG）结构的Agent工作流进行端到端回归测试，帮助团队在CI/CD阶段拦截有缺陷的Agent发布。
- **Galileo**
 - **特点**：支持将离线测试与实时线上护栏（Guardrails）相结合。不仅在发布前评估Agent的轨迹合理性，还能在生产环境中监控Agent是否产生了死循环、越权工具调用或越轨行为。

3.8 Agentic RL (☆☆)

Search-R1

TinyZero

ReTool

Multi-Turn-RL-Agent

DeepResearcher

3.9 Multi-Agent (★)

Here are the main patterns for building multi-agent systems, each suited to different use cases:

Pattern	How it works
Subagents	A main agent coordinates subagents as tools. All routing passes through the main agent, which decides when and how to invoke each subagent.
Handoffs	Behavior changes dynamically based on state. Tool calls update a state variable that triggers routing or configuration changes, switching agents or adjusting the current agent's tools and prompt.
Skills	Specialized prompts and knowledge loaded on-demand. A single agent stays in control while loading context from skills as needed.
Router	A routing step classifies input and directs it to one or more specialized agents. Results are synthesized into a combined response.
Custom workflow	Build bespoke execution flows with LangGraph , mixing deterministic logic and agentic behavior. Embed other patterns as nodes in your workflow.

模拟面试

Q1: skills和mcp的区别?

Q2:

Q3:

四、部署

Python 进阶：异步 asyncio、Pydantic（函数参数结构化，Function Calling 必用）、FastAPI 封装 Agent 服务接口；

向量数据库：Chroma、FAISS、Milvus、Qdrant（熟练建库、插入、检索）；

大模型本地部署：vLLM/SGLang 部署开源模型（Qwen/Llama3/Mistral），对接私有 LLM 做本地 Agent；

容器：Docker 打包 Agent 项目，基础 Docker-Compose。

4.1 LLM推理优化 (★)

量化基础：FP8、GPTQ、AWQ，推理优化基础（vLLM/SGLang/PagedAttention），看懂推理链路。

4.2 服务端部署

- [vLLM](#)
- [SGLang](#)
- mindie

面试高频：

- Prefix Cache
- KV Cache
- Continuous Batching
- Paged Attention
- Speculative Decoding

4.3 端侧部署

1. ONNX Runtime Mobile (ORT Mobile)

- 定位：**跨平台端侧推理首选**，适配 Android/iOS/ 车机 Linux，支持 ONNX 模型
- 优势：体积小、功耗低、算子优化成熟，支持 CPU/GPU/NPU 异构加速
- 适用：绝大多数轻量 LLM、多模态模型，工程落地最稳
- 场景：手机 APP、常规车机座舱

2. TensorFlow Lite (TFLite / TFLite Micro)

- 定位：谷歌端侧推理框架，分标准版 + 微型版
- 优势：生态完善、编译简单，TFLite Micro 适配**极低算力 MCU / 低配车机**
- 局限：大 LLM 算子支持偏弱，更适合小模型、CV、语音
- 场景：低端车机、物联网终端、嵌入式设备

3. llama.cpp (现象级开源)

- 核心：纯 C/C++ 实现，无依赖，极致轻量化
- 亮点：支持 GGUF/GGML 量化 (4bit/8bit)、CPU/GPU 加速，跨平台无敌
- 适配：Android、iOS、Linux 车机、嵌入式 Linux，**车载 / 移动端离线 LLM 首选**
- 用法：可封装 JNI/OC 接口集成进 APP、车机应用，社区模型最全

4.4 端侧 Agent

4.5 标准OpenAI接口

request

response

五、其它

A2A

CodeAct

Skills (★)

Harness

Harness Evolving

AutoPE

Memory Evolve

AlphaEvolve

Hyper-Agents

六、项目

6.1 工作经历

6.1.1 马上消费金融有限公司（2025.10-至今）

A. [对话数据标注自进化Agent系统](#)

简历版

项目名称：对话数据标注自进化Agent系统

技术栈：Agent Framework, LLM, Prompt Optimization, Text Grad, GRPO, Dynamic RAG, LangGraph

项目描述：针对大模型在垂直业务落地中手工 Prompt 调试成本高、质量不稳定的痛点，设计并实现了一套参数无关（Parameter-Free）的推理增强系统，通过在标注过程中动态沉淀领域经验与自动优化 Prompt，实现低成本迭代。

核心工作：

1. 自进化标注与非参数 GRPO 优化：基于 Few-shot 构建分类体系，结合 GRPO 多类别对比机制强化分类边界学习，并引入 TextGrad 对推理轨迹进行分析生成修正规则，沉淀至经验库。在业务中将“电销黑名单”分类准确率从 81.9% → 89.96%， “试驾邀约”意图识别从 58% → 73%，平均提升 8%+。

2. 分层协同与动静隔离架构：构建“主分类 + 易错诊断 + 专有二分类”多 Agent 协同结构，将复杂迭代反思放入异步编译期执行；在线推理仅保留轻量 Prompt + 动态经验检索，实现小时级策略迭代与显著降低人工成本。

3. 防漂移评测与动态经验管理：引入 Golden Set 回归评测防止语义漂移，结合 BM25 + 向量检索的混合召回机制，并对经验库进行周期性去重与合并控制规模，保证系统稳定性与低延迟（单次推理延迟增量控制在秒级）。

模拟面试

<https://chatgpt.com/c/6a2d55bd-b3a8-83ea-9608-e0771a9229e3>

<https://chatgpt.com/c/6a2a4aff-22ac-83ea-bdf4-3631c4dbf48d>

项目名称：对话数据标注自进化Agent系统

项目角色：独立设计与研发负责人

项目背景：针对大模型在垂直业务（如意图识别、复杂多分类、逻辑推理）落地中面临的**手工 Prompt 调试成本高、质量不稳定、领域知识难以持续积累**等痛点，设计并实现了一套无需参数微调（Parameter-Free）的推理增强系统。该系统通过在推理/标注过程中动态沉淀领域经验、自动优化 Prompt，实现了低成本、可解释的性能迭代。

技术栈：Agent Framework, LLM API, Prompt Optimization, Text Grad（文本梯度），GRPO, Dynamic RAG, Workflow Orchestration

核心职责与技术实现

1. **自进化语料标注机制**

- 设计并实现自进化语料标注 Agent，利用少量种子样本（Few-shot Seeds）引导大模型自顶向下构建分类知识体系。
- 建立多维推理链路分析模块，实现对海量对话数据的自动标注与持续优化，解决传统人工标注成本高、迭代慢的问题。

2. **非参数化 Multi-class GRPO 与 Text Grad 协同优化**

- 范式扩展**：将 GRPO（群体相对策略优化）的“单样本-多轨迹”对比机制适配至提示词优化场景。通过在单个 Batch 中引入多类别样本，引导模型对跨类别的推理轨迹进行对比“多样本-多轨迹”，显式提炼分类边界。

- ****文本梯度反思****: 基于 **Text Grad** 思想, 通过 **Agent** 自省分析正逆向推理轨迹的差异, 生成文本级“梯度修正意见”, 并将提炼出的泛化规则以结构化 **JSON** 格式沉淀至动态经验库。

- ****自适应反馈调节****: 设计与 **Batch** 准确率挂钩的自适应奖励函数。在低准确率区间降低更新门槛（侧重探索与快速修正），在高准确率区间收紧更新门槛（侧重利用与系统稳定），平滑引导 **Prompt** 与经验库的协同演进。

3. ****分层 Agent 协同架构与动静隔离优化****

- 构建由“多分类主 **Agent** + 易错类别二分类诊断 **Agent**”组成的分层协同闭环。

- ****运行期与编译期解耦****: 将重资源的 **Multi-class GRPO** 迭代、**Text Grad** 反思等步骤放在异步“编译期”运行；线上真实推理（运行期）则采用“精简版 **Prompt** + 动态检索关联经验”的轻量化架构，降低系统延迟与 **Token** 消耗。

4. ****量化评测门禁系统与防漂移机制****

- ****黄金验证集（Golden Set）防漂移门禁****: 为防范自进化过程中的“语义漂移”与“灾难性遗忘”，建立自动化评测 **Pipeline**。每次 **Prompt** 或经验库迭代更新前，强制在黄金验证集上进行回归测试，指标未发生退化方可准入上线。

- ****经验库动态剪枝与路由****: 引入基于向量与 **BM25** 的混合检索机制，在线推理时仅按需召回 **Top-K** 相关规则；同时设计定期合并与去重机制，设置近一段时间被检索次数，为0则人为删除，保持经验库的规模处于合理区间。

****效果与业绩****

- ****多场景准确率显著提升****: 方案在多个实际业务场景中完成落地验证。其中，“电销黑名单”分类准确率由 ****81.9%** 提升至 **89.96%****，“试驾邀约”意图识别准确率由 ****58%** 提升至 **73%****；多数主流场景的标注准确率平均提升 ****8%** 以上，标注一致性与标准化程度得到明显改善。

- ****具备鲁棒的自诊断与偏差发现能力****: 在“质检分类”等未见明显指标提升的复杂场景中，利用系统的“易错类别发现 **Agent**”与自评估机制，主动诊断并定位出评估数据源自身存在的标注偏差，避免了算法盲目拟合噪声，验证了系统评估体系的客观性与严谨性。

- ****标注效率跃升与人工降本****: 通过全自动化的 **Prompt** 与经验库迭代流程，将策略调优周期由传统人工的****数天**缩短至**小时级**；大幅减少了标注过程中的人工干预和校验频次，显著提升了海量数据的整体标注吞吐量。

- ****高工程性能与泛化扩展性****: 通过引入动态经验库压缩机制，单次推理延迟增量控制在极低水平（毫秒级），满足高并发标注的性能要求；系统分层架构具备良好扩展性，可无缝复用至 **RAG**、**Agent** 等更复杂的上下文学习（**In-Context Learning**）场景。

训练阶段 **Text Grad** 生成 **m** 个 **prompt** 模板，给每个样本使用，**batch = group(n) * prompt(m)**

面试时的口语化讲法:

我做的是一个面向对话语料标注的自进化智能体系统。这个系统的目标是解决人工标注贵、慢，而且遇到新业务和新表达时很难快速适应的问题。整体上分成三层：先用通用粗分类做初始标注，再根据易错类别做自进化优化，最后通过细分类模块对高混淆样本逐级校验。系统还会自动优化提示词，让大模型在不训练参数的情况下持续提升标注效果。

模拟问题

Q1: 为什么选择"无训练"方案而不是直接微调模型?

回答要点: 微调成本高（算力、数据标注）、周期长、难以快速迭代；无训练方案通过推理阶段积累经验，零算力成本、实时可更新，适合资源受限的业务场景。可对比 **RAG**（检索外部知识）与本方案（自生成内部经验）的差异。

- 原因1: 场景变化太快(SFT 周期非常长)
- 原因2: 数据规模不足(SFT 500~3000条标注数据)
- 原因3: 解释性要求高(SFT 难解释)

Q2: 为什么选择 TextGrad 框架? 它们的核心差异是什么?

回答要点: TextGrad 将"文本梯度"概念引入优化, 适合需要多跳推理的复杂任务。

Q3: 经验库如何避免"噪声经验"污染后续推理?

回答要点: 通过两阶段机制: ① 单 Batch 内先生成 candidate_experiences 而非直接更新, 避免单条坏样本直接写入; ② Epoch 结束后的压缩阶段通过 LLM 二次审查合并冗余、淘汰低质量条目。可进一步讨论设置置信度阈值等改进方向。

Q4:

Q5: 经验的"增删改合并"操作由 LLM 自身决策, 如何保证 JSON 格式输出的鲁棒性?

回答要点: 代码中使用 `split('```json')[-1].split('```')[0]` 做格式抽取, 外层 try/except 兜底, 失败时跳过而非崩溃 (continue)。可进一步讨论引入 Pydantic 校验、Few-shot 格式示例强化输出稳定性

Q7: 如何评估先验经验的质量与有效性?

回答要点: 当前方案以最终答案正确率作为隐式反馈; 可扩展讨论: 引入显式的经验有效性评分 (如某条经验在多少问题上有帮助)、A/B 测试 (加/不加经验的准确率对比)。

双路消融对比:

- **方法:** 在同一验证集 (Validation Set) 上运行两路 Pipeline:
 - **对照组 (Base Line):** 仅使用原始 Prompt, 不引入 JSON 经验库。
 - **实验组 (Exp Line):** 在输入端注入当前版本的 JSON 经验库。
- **评估指标:** 计算 **准确率** 和 **F1-Score** 增量。
。若某条经验加入后 $\Delta Acc < 0$, 说明该经验存在负迁移, 应予以剔除或降级。

留一交叉验证:

1. 留一交叉验证 (Leave-One-Out Validation) :

- **方法:** 针对经验库中的多条经验 $E=\{e_1, e_2, \dots, e_n\}$, 每次有意识地移除其中一条 e_i , 观察整体表现的变化。
- **评估指标:** 计算经验的**边际贡献度**。边际贡献为负或接近于零的经验会被标记为“低效经验”, 触发清理机制。

Q8: 新沉淀的经验与既有经验产生逻辑冲突, 如何解决?

1. 语义冲突检测 (Conflict Detection) :

- **方法:** 利用专有的“审计 Agent”或符号化规则, 对新生成的 JSON 经验与既有经验进行两两比对。
- **评估案例:** 若经验 A 指出“退款”关键词属于“售后服务”, 而新经验 B 指出“退款”且伴随抱怨语时应归入“投诉”, 系统需评估两者的覆盖边界。

- **解决机制**：若判定为冲突，则触发“规则融合（Merge）”或“条件细化（Refinement）”，通过补充前置条件（如 Context-specific constraints）来消解冲突。

2. 语义冗余度与合并（Redundancy & Deduplication）：

- **方法**：计算新经验与旧经验在向量空间（Embedding）中的余弦相似度。
- **评估指标**：当相似度超过设定阈值（如 0.85）时，判定为表述冗余。系统将自动调用大模型进行“规则归并”，用更具泛化性的表述合并这两条经验。

Q9：如何提升跨样本的泛化能力？

回答要点：高温度增加输出多样性，保证 N 条采样轨迹覆盖不同推理路径（正确与错误），为经验提取提供更丰富的对比素材。若 temperature 过低，N 条输出高度相似，经验提取价值有限。

1. 跨批次泛化测试：

- **方法**：将在批次 T 中提炼出的经验，立即在完全独立的批次 T+1、T+2 上进行盲测。
- **评估指标**：**泛化衰减率。如果在提炼批次上准确率提升 15%，但在新批次上仅提升 1%，说明该经验过拟合了特定批次的语料（如特定客户的说话口癖），属于低质量经验。

2. 生存周期与置信度衰减：

- **方法**：给每条 JSON 经验赋予一个初始置信度（Confidence Score）和调用计数器。
- **评估机制**：在后续标注中，每当某条经验被成功检索并调用：
 - 若导致标注正确：置信度增加（奖励）；
 - 若导致标注错误：置信度大幅扣减（惩罚）。
 - 当置信度低于阈值，或者长期未被调用，该经验将被自动物理删除或归档，以保持经验库的“代谢活性”。

3. 引入“黄金验证集”的门禁机制：

- **方法**：在每次经验库或 Prompt 更新后，不能仅依赖当前 Batch 的准确率，还需自动在代表性历史样本集上进行微调级别的回归测试。

Q10：temperature=1.0 的设计意图是什么？

回答要点：高温度增加输出多样性，保证 N 条采样轨迹覆盖不同推理路径（正确与错误），为经验提取提供更丰富的对比素材。若 temperature 过低，N 条输出高度相似，经验提取价值有限。

Q11：这套系统如何从实验原型落地到生产环境？你会做哪些工程化改造？

回答要点：① 经验库改为向量数据库（如 Chroma，代码中已有依赖）实现语义检索而非全量注入；② 增加经验版本管理与回滚机制；③ 异步任务队列处理大批量数据；④ 监控 API 调用成本与经验库增长曲线。

Q12：如果业务方反馈模型在新数据上效果下降如何排查？

回答要点：① 检查经验库是否存在过拟合训练分布的噪声经验；② 对比加/不加经验的推理结果，定位经验质量问题；③ 检查数据分布漂移（新数据与训练数据的 prompt 模式差异）；④ 必要时清空经验库重新从新数据中提取。

Q11: 先介绍一下这个项目。不要讲技术细节，重点讲业务背景、为什么要做、最终解决了什么问题。

Q12:

Q13: 你提到把GRPO改造成Prompt优化。详细讲一下。

传统GRPO: 优化对象是模型参数。

Multi-Class GRPO:

把优化对象从: Model weight

变成: Prompt + 经验库规则

下一轮推理时:

动态召回。效果逐步提升。

Q14: Multi-Class GRPO和普通GRPO区别是什么?

普通GRPO:

同一问题

多个轨迹比较

Multi-Class GRPO:

每个batch中包含多个类别

跨类别边界比较

Hard Negative

→发现问题

TextGrad

→总结规律

经验库

→长期记忆

Q15: 为什么不在线学习?

1. 稳定性不可控。

2. GRPO+TextGrad 一次优化需要几十到上百次LLM调用，线上无法接受。

Q16: 经验库越来越大怎么办?

第一层: Embedding召回，经验向量化，TopK=10

第二层: BM25召回，关键词补充，防止向量漏召回

第三层: 规则聚合，规则合并

第四层: 检索次数少的去掉，且创建时间较早，去除老旧经验

Q17: 线上出现过最严重的问题是什么？

某批数据本身标注错误。
系统把错误样本当真理论学习。
生成大量错误规则。

Q18: GRPO的奖励函数如何设计的？

第一步：考核候选提示词的“三个指标”
 答得对不对（正确率得分）
 答得清不清晰（边界区分得分）
 是不是太啰嗦（字数冗余扣分）

第二步：同台竞技，只看“相对优势”（GRPO 核心）
 每次迭代，系统会同时生成 5~10 个不同的提示词版本。
 让这些版本在同一批测试数据上“同台竞技”，算出各自的总分。
 计算它们的平均分。
 比平均分高得多的，定义为***“尖子生（正向反馈）”；比平均分低得多的，定义为“不及格生（逆向反馈）”**。
 文本梯度系统只会参考“尖子生”和“不及格生”的答题差异来进行针对性反思。

第三步：自适应调整考核重点（动态风控）
 模式 A：低分低谷期（系统还很笨的时候，比如正确率低于 60%）
 考核重点：只看正确率。只要能答对，逻辑粗糙点、提示词长一点都没关系。
 更新门槛：降到最低。只要有稍微好一点的修改，立刻通过，鼓励系统大胆探索。
 模式 B：高分平稳期（系统已经很聪明的时候，比如正确率高于 85%）
 考核重点：转向边界清晰度和字数精简度。正确率高已经不稀奇了，我们要看谁的推理逻辑最干净、字数最少。
 更新门槛：提到最高。只有能带来显著突破且不啰嗦的修改才允许上线，防止系统“瞎折腾”导致线上不稳定。

Q20: 动态经验库的“规则冲突”与一致性如何维护？

1. 编译期冲突检测与消解机制（Consistency Checking）：
- 当有新规则拟写入经验库时，审计 Agent 负责检索库中语义相似的已有规则，并利用大模型进行一致性推理。
 - 若发现冲突，进行“规则合并（Consolidation）”，融合成一条描述更精确、边界更清晰的上位规则，或者通过显式声明优先级（e.g., if A then X, unless B）来解决。
2. 规则的时效性衰减（Time-decay / Usage-decay）：
- 为每条规则维护一个使用计数和准确率反馈指标。如果某条规则长期未被检索，或者被检索后导致线上推理判断失误，则降低其置信度权重，直至触发动态剪枝。

Q21:

简历版

项目名称：大模型零幻觉推理研究

****技术栈**:** Python、PyTorch、Qwen、RAG、FAISS、RAGAS、Agent、vLLM

****项目描述**:** 面向大模型事实幻觉与推理幻觉问题，研发基于分层检索增强、推理验证和自我反思机制的零幻觉推理框架，提升复杂问答与Agent场景下的可信推理能力。

****核心工作**:**

构建 **Agentic RAG** 引擎：设计三级轻量索引与 **ReAct** 动态路由机制，实现多粒度检索与证据链的自主收集。

设计多阶段推理流水线：搭建“优化-检索-推理-验证-反思”五阶段架构，支持复杂问题的多轮联动推理与证据补全。

实现闭环自纠错机制：融合验证链（CoV）与一致性投票，引入反思智能体，形成“幻觉识别-补全-重推理-验证”的纠错闭环。

搭建自动化评测方案：基于公开基准完成消融实验，构建自动化评测流水线以支撑系统的持续迭代。

****项目名称**:** 大模型零幻觉推理研究

****技术栈**:** Python、PyTorch、Transformers、RAG、FAISS、LoRA、RAGAS、Graph of Thoughts、Self-RAG、Self-Consistency、Self Reflection、Claim Verification、Cove (Chain of Verification)、Agent、Prompt Engineering

****项目描述****针对大语言模型在知识问答、复杂推理及 Agent 场景中存在的事实幻觉（Factual Hallucination）和推理幻觉（Reasoning Hallucination）问题，同时解决传统静态 RAG 检索策略固化、上下文利用率低、无法自适应复杂多跳推理的痛点，引入 ****Agentic RAG（智能化检索增强）**** 范式，构建「分层检索增强 + 多路径推理 + 链式事实验证 + 自我反思纠错」的全链路零幻觉推理框架。通过引入动态知识检索、多路径推理、答案验证及可信度评估机制，实现模型回答过程的可追溯、可验证与可纠错，显著提升复杂任务场景下的事实准确率与推理可靠性。

****核心工作****

1. 智能化分层检索引擎设计与实现

提出并落地分层检索接口式 **Agentic RAG** 系统，构建「关键词级 - 句子级 - 语块级」三级轻量索引体系，对应开发**keyword_search**精确词法匹配、**semantic_search**句子级稠密语义检索、**chunk_read**按需全文读取三类检索工具，赋予大模型自主选择检索粒度、编排检索流程的能力，替代传统单轮静态检索范式。

基于 **ReAct** 框架实现智能体检索循环，配套上下文追踪机制避免重复读取语块，实现渐进式信息获取；底层基于 **FAISS** 构建句子级向量索引，结合实时精确匹配实现混合检索，兼顾检索精度与查询效率。

设计自主路由机制，由模型自主判断是否需要检索、选择何种检索方式、何时终止检索并输出答案，支持多跳复杂问题的自主分解与分步证据收集，自适应不同难度的问答任务。

2. 多阶段零幻觉推理链路构建

搭建查询改写 → 自主检索 → 多路径推理 → 事实验证 → 自我反思五阶段推理流水线，实现检索与推理的深度交替联动；融合 **Graph of Thoughts** 多路径推理思想，支持复杂问题的拆解与并行验证。

结合 **Self-RAG** 自评估思路，让模型在推理过程中主动判断知识缺口并触发定向检索，而非依赖外部规则触发，从源头减少参数知识偏差导致的幻觉。

支持多轮迭代补全证据链，针对验证环节发现的事实缺失、逻辑断点，自动触发二次检索补充信息，解决单轮检索信息不足导致的隐性幻觉。

3. 推理验证与自纠错闭环机制

引入CoVe（验证链）与 Self-Consistency 多路径投票机制，对模型输出开展逐事实核验与逻辑一致性校验，精准定位推理链中的实体混淆、策略偏差、逻辑跳变等隐性错误。

实现 Reflection Agent 反思模块，对低置信度、事实冲突的回答自动触发检索策略调整与二次推理，结合分层检索工具定向补充缺失证据，形成「幻觉检测 → 检索补全 → 重推理 → 二次验证」的自优化闭环。

构建「规则校验 + LLM Judge」双维度幻觉检测模块，覆盖无依据生成、事实冲突、推理谬误等多类幻觉场景，实现错误的精准分类与定位，为检索与推理策略迭代提供数据支撑。

4. 可信评测体系搭建与规模化验证

基于 RAGAS、HaluEval、HalluQA 及 HotpotQA、MuSiQue 等多跳问答基准构建评测方案，覆盖 Faithfulness、Answer Correctness、Groundedness、幻觉率、上下文效率等多维度指标。

完成系统消融实验与规模化验证：证实三类分层检索工具组合达到最优效果，相比传统单轮 RAG，在多跳问答任务上准确率显著提升，同时检索 token 量降低约 30%；验证框架具备测试时可扩展性，随推理步数增加与基座模型能力提升，性能持续稳定增益。

搭建自动化评测流水线，支持检索策略、Prompt 模板、验证机制的快速迭代优化，支撑框架的持续迭代。

模拟问题

Q1：基础介绍

Q2：你们怎么定义“幻觉”？是怎么量化的？

1) 事实幻觉 (Factual Hallucination)

定义为：Answer 中的关键事实与可信知识源 (retrieval / dataset / knowledge base) 不一致

量化方式：

是否有 unsupported claim (RAGAS)

是否存在 retrieval mismatch

是否无法在 evidence 中找到支撑

2) 推理幻觉 (Reasoning Hallucination)

定义为：推理链中任意 step 逻辑不成立，但最终答案可能看似正确

量化方式：

step-level verification (CoVe)

self-consistency disagreement rate

judge model 对 reasoning chain 的打分

Q3：claim 是怎么拆的？是规则拆还是 LLM拆？会不会不稳定？

我们用了混合方式：

1) 规则拆（轻量场景）

按句号 / 逻辑连接词拆句

适合 FAQ / 短文本

2) LLM Claim Extraction（主方案）

用 prompt 让模型输出：

atomic claims（原子事实）

每个 claim 对应 evidence query

Q4: Query Rewrite → Retrieval → Reasoning → Verification → Reflection, 这个链路是串行的吗? 还是有并行? 为什么这样设计?

整体是主链路串行 + 局部并行优化:

主链路:

Query → Rewrite → Retrieval → Reasoning → Verification → Reflection

原因:

幻觉问题本质是“逐步放大错误”

必须强制 step-wise gating

并行优化点:

1) Retrieval 并行

BM25 + Dense Retrieval 同时跑

FAISS + keyword index hybrid

2) Reasoning 多路径并行 (Self-Consistency)

多个 CoT sampling

最后 vote

Q5: 你这个链路 latency 怎么控制? 不会很慢吗? 线上怎么用?

1) Router 机制 (关键)

不是所有请求都走全链路:

简单问题 → 直出 LLM

知识类 → RAG

高风险 → 全链路验证

👉 用 Agent Router 做 classification

2) Early Exit (提前终止)

如果:

retrieval confidence 高

reasoning consistency 高

👉 直接跳过 Reflection

3) 缓存机制

retrieval cache (query embedding)

reasoning cache (similar query patterns)

Q6:

Q7: CoVe 和普通 RAG 最大区别是什么?

普通 RAG:

“找资料 → 直接回答”

CoVe:

“先生成 → 再逐条验证 → 再修正”

Q8: CoVe 验证用的是什么模型? 如果还是同一个 LLM, 不是自嗨吗?

1) Same Model Self-check (弱)

同一个 LLM

prompt: “检查是否有错误”

优点: 便宜

缺点: 容易自我强化错误

2) Cross Model Verification (中)

generator ≠ judge

用 Qwen / GPT / Claude 不同模型交叉验证

3) Retrieval-grounded verification (强)

每个 claim 必须匹配 evidence

用 embedding similarity + NLI judge

最终采用:

“模型 + 检索 + NLI 三重验证”

Q9: Self-RAG 和 CoVe 有什么本质区别? 你为什么两个都用? 不是重复吗?

区别在“发生时机不同”, 解决不同问题:

CoVe:

post-hoc verification (事后检查, 修正已生成错误)

Self-RAG:

generation-in-loop (生成过程中动态检索, 防止生成过程跑偏)

Q10:

Q11: 最难优化的是哪个模块? 为什么?

最难的是:

Reflection Agent (自反思模块)

问题在于:

1) 容易“过度纠错”

正确答案被误判为错误

2) 不稳定

同一问题不同运行结果不同

解决方案：

- 1) 引入置信度阈值
`confidence > 0.85` → 不触发 `reflection`
- 2) 规则 + LLM hybrid judge
规则过滤明显正确答案
LLM只处理边界case

Q12：如果让你重做这个项目，你会砍掉哪个模块？为什么？

我会考虑：

砍掉“过重的 `multi-path Self-Consistency`”

原因：

- 成本太高（推理次数线性增长）
- 在很多企业QA场景收益有限
- `Cove + retrieval` 已经覆盖大部分收益

优化方向会变成：

“轻量验证 + 强检索 + `selective reasoning`”

Q13：“轻量”的核心体现

- 1、相比 `GraphRAG` 需要做实体抽取、关系抽取、图谱构建、社区聚类等重型离线计算，该方案无图结构、无复杂预处理；
- 2、关键词层不提前构建倒排索引，运行时实时匹配，大幅降低离线索引的时间与算力成本；
- 3、仅需「语块切分 + 句子嵌入」两个核心步骤，工程落地成本低、扩展性强。

Q14：三级轻量索引：底层数据结构的分层设计

索引层级	存储内容	构建方式	核心作用
关键词级 (隐式层)	语块的原始全文文本	离线不做预索引，查询时实时做精确文本匹配	支撑精确实体、专有名词的快速定位
句子级（语义层）	每个单句的稠密向量 + 句子→所属语块的映射关系	对每个语块分句，逐句计算嵌入向量	支撑细粒度语义匹配，精准定位相关句子
语块级（全文层）	1000token 左右的完整语块文本 + 元数据	按句子边界切分语料，保证语义完整	作为最终读取单元，提供完整上下文

Q15：三类分层检索工具

检索工具	对应索引层级	工作机制	模型使用场景
<code>keyword_search</code> 关键词检索	关键词级（隐式层）	基于精确词法匹配，按「关键词出现次数 × 关键词长度」打分，返回包含关键词的句子片段	已知明确实体、专有名词、数字、日期，需要精准定位语块时
<code>semantic_search</code> 语义检索	句子级（语义层）	基于句子向量做余弦相似度匹配，按语块内最高相似度句子打分，返回匹配句子片段	关键词检索失效、需要概念性 / 模糊语义匹配时
<code>chunk_read</code> 语块读取	语块级（全文层）	根据语块 ID 返回完整文本，配合上下文追踪器避免重复读取	通过前两类工具筛选出高相关语块后，读取全文做深度推理

Q16：分层检索的完整 workflow

三级索引是“数据底座”，三类工具是“访问入口”，二者一一对应，共同实现**渐进式、按需式的分层检索**，这也是和传统单轮 RAG 最核心的区别：

- 1. 模型先通过 `keyword_search` 或 `semantic_search` 做**粗筛**：只返回句子级片段，用极少 token 快速判断哪些语块可能相关；
- 2. 模型自主判断相关性后，再通过 `chunk_read` 做**精读**：只对高相关语块读取完整全文，避免大量无关内容占用上下文；
- 3. 若信息不足，模型可自主切换检索工具、调整关键词 / 查询，反复迭代补全证据链，直到信息足够再输出答案。

简单来说：**三级索引定义了“有什么信息可以用”，三类工具定义了“模型怎么用这些信息”，二者结合实现了“检索粒度由模型自主决定”，这就是 Agentic RAG 的核心体现。**

Q17：关键字检索的整体流程



Q18: 整体架构: 「主干流程 + 质量闭环」的双层嵌套模式

查询优化 → 自主检索（内层A-RAG自主迭代） → 多路径推理 → 事实验证（幻觉检测） → 自我反思（纠错决策）

[illegible]

五阶段推理链路是任务从输入到输出的标准执行主干，规定了信息处理的完整顺序；推理验证与自纠错闭环是内嵌在链路中的质量管控机制，以「事实验证」为检测入口、以「自我反思」为决策中枢，可根据错误类型回退到任意前序阶段触发修正。

二者不是两套并行系统，而是形成「单向主流流转 + 定向回退修正」的嵌套结构，同时在内层嵌套 A-RAG 的检索 - 推理自主迭代，最终实现“一次输入、多层校验、动态纠错”的零幻觉效果。

Q19: 阶段一：查询优化

对原始问题做结构化拆解：识别问题类型（单跳 / 多跳 / 开放域）、拆分多跳子问题、生成适配不同检索工具的查询表达式，输出结构化的子问题列表与对应检索 Query，为后续检索和推理明确方向。

与自纠错闭环的联动

- 首次执行时，按默认拆解逻辑运行，是全链路的起点；
- 接收反思阶段回退的「查询修正指令」：当后续验证发现问题理解偏差、子问题拆解错误、检索方向完全跑偏时，反思阶段会输出具体的重写要求（如“补充时间限定条件”“将复合问题拆分为两个独立子问题”），本阶段根据指令重新改写查询，再从头走后续链路。

输出产物：结构化子问题列表、对应检索关键词 / 语义查询、问题约束条件

Q20: 阶段二：自主检索 (Autonomous Retrieval)

基于 A-RAG 分层检索框架执行证据收集，这是**内层的第一级自迭代循环**：大模型通过 ReAct 模式自主调用三类分层检索工具（`keyword_search` 精确匹配、`semantic_search` 语义匹配、`chunk_read` 全文读取），自主判断信息是否充足、自主切换检索策略，直到认为证据可支撑推理，再进入下一阶段。

这个过程本身就是检索层面的自纠错——模型会自主修正检索关键词、更换检索工具、补充读取语块，无需外部规则触发。

与自纠错闭环的联动

- 接收反思阶段回退的「检索补全指令」：当验证发现证据缺失、事实冲突、实体信息不全时，反思阶段会输出定向检索要求（如“精确匹配 XX 实体的任职时间”“补充 XX 事件的背景上下文”）；
- 本阶段根据指令定向执行检索：优先用关键词检索锁定目标语块，读取全文补充证据，必要时读取相邻语块完善上下文，完成证据补全后直接送入推理阶段重算，无需回到查询改写阶段，最小化重复计算。

输出产物：带来源标注的证据语块集合（含 chunk ID、原文片段、检索方式）、已读语块追踪记录。

Q21: 阶段三：多路径推理 (Multi-path Reasoning)

基于已收集的证据集，融合 Graph of Thoughts 与 Self-Consistency 思想，生成多条独立的推理路径，每条路径完整推导答案并标注证据支撑点，最终输出多组候选答案与对应的推理链。

与自纠错闭环的联动

- 接收反思阶段回退的「推理修正指令」：当验证发现逻辑谬误、推理跳步、路径偏差但证据本身充足时，反思阶段会输出修正方向（如“补充第二步的推导依据”“排除与证据冲突的推理分支”）；
- 本阶段根据指令调整推理路径、补充逻辑环节、更换推导思路，重新生成推理链与候选答案，再送入验证阶段，无需回退到检索环节。

输出产物：多组推理链、对应候选答案、每条路径的初步置信度。

Q22: 阶段四：事实验证 (Verification) —— 闭环的检测入口

这是自纠错闭环的**核心检测层**，只负责“发现问题、定位错误”，不做修正决策，是连接主流程与闭环的关键节点。

主链路核心动作

对推理阶段的输出做三维度全量校验，输出结构化错误报告：

- 事实性校验 (CoVe 验证链)：**将最终答案拆解为独立的事实断言，逐一与证据库中的原文做比对，标记出无依据断言、事实冲突断言、模糊表述断言，定位到具体缺失的信息点。
- 一致性校验 (Self-Consistency)：**对多条推理路径的答案与逻辑做一致性投票，统计答案分歧点、逻辑矛盾点，标记低共识的推理环节。
- 双维度幻觉检测：**结合规则校验（如实体匹配、数值校验）与 LLM Judge，识别实体混淆、无中生有、逻辑跳步、因果倒置等具体错误类型，评估错误严重程度。

与自纠错闭环的联动

- 所有校验结果汇总为结构化错误报告，包含：错误类型、错误位置、严重等级、缺失 / 冲突的具体信息点，直接输入到自我反思阶段，作为纠错决策的唯一依据；
- 本阶段不干预主流程走向，只输出客观检测结果，决策完全由反思阶段完成，实现“检测与决策分离”。

输出产物：结构化幻觉检测报告、事实一致性得分、推理置信度评分。

Q23: 阶段五：自我反思 (Reflection) —— 闭环的决策中枢

这是自纠错闭环的**指挥中枢**，负责根据验证报告做最终决策：是直接输出答案，还是触发回退修正，是整个闭环的核心控制单元。

Reflection Agent 接收验证报告，结合当前迭代次数，做出三类决策：

- 置信度达标：**所有断言均有证据支撑、推理逻辑自洽、一致性达标，直接输出最终答案，全链路结束；
- 置信度不达标且未达最大迭代次数：**根据错误类型，生成精准的修正指令，回退到对应前序阶段触发重跑；
- 达到最大迭代次数：**强制终止闭环，基于已有信息输出带置信度标注的答案，避免无限循环。

输出产物：最终答案（带证据引用） / 定向修正指令 + 迭代次数记录。

Q23:

Q23:

#####

6.1.2 浪潮通信 (2021.07-2025.10)

大模型训推与异构算力适配平台 → 通用通信大模型训练 → 智能体平台；

A. 大模型训推与异构算力适配平台

简历版

****项目名称**:** 大模型训推与异构算力适配平台

****技术栈**:** Python、CUDA、CANN、MindIE、vLLM、SGLang、FastAPI、Docker、LoRA/QLoRA、模型蒸馏、LLM

****项目描述**:**
搭建覆盖数据处理、模型训练、模型蒸馏、推理部署及模型管理的一站式大模型平台，支持GPU、NPU（昇腾910B）、DCU等异构算力环境，实现大模型、多模态模型、Embedding及Reranker模型的统一训推与部署。累计完成10+模型跨平台适配，模型交付周期由3天缩短至1天以内，整体效率提升60%以上。

****核心工作**:**
1.数据与训练平台建设：自研数据标注与蒸馏工具，构建高质量训练数据流水线；封装PT、SFT、DPO、蒸馏等训练范式，支持Full Fine-tuning、LoRA、QLoRA等微调方案。
2.异构算力适配：完成GPU、NPU及DCU环境适配，解决模型迁移适配、训练推理可用性验证、多机多卡性能测试和瓶颈分析。
3.推理与平台化建设：集成vLLM、SGLang、MindIE等推理引擎，基于FastAPI与Docker实现模型工程化部署落地。

模拟面试

****项目名称**:** 大模型训推与异构算力适配平台

****技术栈**:** Python、CUDA、CANN、MindIE、vLLM、SGLang、FastAPI、Docker、LLMs、LoRA/QLoRA、模型蒸馏、Embedding、Reranker、多模态模型

****项目描述**:**
搭建面向企业级AI应用的一站式大模型训推与异构算力适配平台，覆盖数据处理、模型训练、模型蒸馏、推理部署、模型管理及跨硬件适配全生命周期能力。平台底层对接kubernetes实现任务调度与资源编排，上层屏蔽GPU / NPU（昇腾910B）/ DCU等异构算力差异，统一提供模型训练与推理API能力。支持LLM、多模态模型、Embedding、Reranker及语音模型统一纳管，累计完成10+模型跨平台迁移与生产部署，将模型交付周期由3天缩短至1天以内，推理部署从1天缩短至30分钟，整体研发与运维效率提升60%以上。

****核心工作**:**

1. 数据工程与模型蒸馏体系建设

构建“数据生成 → 清洗 → 质量评估 → 回流优化”的闭环数据流水线，实现训练数据自动化生产
基于教师模型（LLM）构建CoT级知识蒸馏机制，自动生成高质量问答对与推理链数据
设计多维数据质量评估指标（语义重复度、困惑度、任务一致性），提升数据有效性
将低质量人工标注依赖降低 50%+

2. 多范式训练平台研发

统一抽象 PT / SFT / DPO / PPO/ GRPO/ KD 四类训练范式，构建可插拔训练执行框架
支持 Full Fine-tuning / LoRA / QLoRA 自动策略选择（基于显存与任务类型）
设计训练任务编排系统（Pipeline化），支持断点恢复、失败重试与任务优先级调度
实现训练过程指标可观测（loss / grad norm / lr / GPU util）

3. 异构算力适配与国产化迁移

在昇腾910B环境下完成浪潮、百川等大模型的推理与微调迁移
基于MindIE/MindFormers解决模型加载、权重转换及训练推理兼容问题，支持LoRA/SFT等训练验证
完成单机多卡与多机多卡性能测试与瓶颈分析

4. 高性能推理与服务部署优化

集成vLLM、SGLang、MindIE等主流推理引擎，实现不同硬件环境下推理框架统一调度；
优化 KV Cache / batch 合并 / continuous batching 提升吞吐能力

5. 模型资产管理与平台化建设

构建统一 Model Registry（LLM / Embedding / Reranker / 多模态）
提供 FastAPI 标准化服务接口 + Docker 化部署链路
对接 Kubernetes（作为底层资源调度系统，非核心自研部分）

项目成果

支撑GPU、NPU、DCU三类异构算力环境统一管理；
完成10+模型跨平台适配与生产部署；
模型交付周期由3天缩短至1天以内；
模型部署周期由1天缩短至30分钟；
运维成本降低50%，整体研发与交付效率提升60%以上；
支撑多个企业级AI应用场景快速落地与迭代。

模拟问答

Q1: 你这个平台和 Kubeflow / MLflow / SageMaker 有什么区别？

我们不是简单的MLOps工具替代，而是做了**“大模型训推一体 + 异构算力适配”的工程抽象层”。

三点核心差异：

1. 面向大模型优化，而不是传统ML

支持 LoRA / QLoRA / DPO / PPO / GRPO
支持 KV cache / continuous batching 推理优化

2. 异构算力统一抽象

GPU（CUDA）+ NPU（CANN/MindIE）+ DCU统一调度
解决算子级别兼容问题，而不是任务级调度

3. 训推闭环

MLflow只管tracking
我们是：数据 → 训练 → 蒸馏 → 推理 → 灰度发布

Q2: 你说支持 DPO / PPO / GRPO, 这些你真的实现了吗?

不是从零实现算法本身, 而是做了**训练范式工程封装 + pipeline标准**:

PPO / DPO / GRPO 使用开源框架 (如 TRL / DeepSpeed-Chat)
我们做的是:

数据格式统一 (prompt / preference / reward)
loss wrapper 封装
多训练策略调度器

GRPO部分:

本质是 group-based reward optimization
我们在 reward ranking 层做了 batch-level grouping

追问兜底:

GRPO和PPO区别你用一句话讲清?
reward model是谁训练的?
reward scaling怎么做?

Q3: 为什么不用全量微调?

✅ 标准回答:

核心是三点工程约束:

1. 显存成本

7B模型 full FT 需要 80GB+
LoRA 只需 20~30%

2. 训练效率

QLoRA 4bit quant + LoRA adapter
提升训练吞吐 2~3倍

3. 多任务适配

adapter可插拔
支持多业务场景共享 base model

追问兜底:

NF4为什么比 INT4更适合LLM?
LoRA rank怎么选?
merge adapter 会不会掉点?

Q4: vLLM 和 SGLang 为什么能提升吞吐?

vLLM核心:

PagedAttention → KV cache 分页管理
避免显存碎片化

SGLang:

structured generation + graph execution
减少 token-by-token overhead

我们平台:

按模型类型自动选择引擎
decoder-only → vLLM
tool-use / structured → SGLang

追问兜底：

KV cache 为什么是瓶颈？

continuous batching怎么实现？

为什么不是 **TensorRT-LLM**？

Q5：你的推理优化做了什么？

三个层面：

算子层

fused attention

RMSNorm融合

调度层

continuous batching

dynamic batch size

缓存层

KV cache reuse

prefix caching

效果：

QPS 提升 **2.5x**

P95 latency 降低 **40%**

追问：

batch size动态怎么调？

OOM怎么防？

GPU利用率怎么测？

Q6：CUDA → CANN 迁移你怎么做的？

主要三步：

算子映射

CUDA kernel → **CANN operator**

不支持的用自定义算子补

图编译适配

PyTorch → **ONNX** → **MindIE graph**

精度对齐

FP16/BF16误差控制

追问兜底：

哪些算子最难迁移？

attention怎么适配？

昇腾性能瓶颈在哪？

Q7: GPU和NPU结果一致性怎么保证?

1. 数值层

FP16/BF16统一精度策略

2. 计算图层

固定seed + deterministic ops

3. 输出校验

KL divergence < threshold

token一致率评估

🔥 追问:

attention softmax误差怎么处理?

哪些任务最容易 drift?

Q8: 你说自动生成数据, 怎么防幻觉?

三重过滤机制:

rule filter

regex / schema validation

model judge

用 teacher model 做 consistency check

embedding filter

去除低相似 / off-topic sample

🔥 追问:

teacher hallucination怎么办?

数据质量怎么量化?

Q9: 蒸馏具体怎么做?

我们做的是:

logit distillation + instruction distillation

loss = CE + KL(student || teacher)

重点优化:

引入 CoT distillation (只保留 reasoning skeleton)

降低冗余推理路径

🔥 追问:

CoT会不会泄露噪声?

student capacity不够怎么办?

Q10: 训练任务失败怎么恢复?

✅ 标准回答:

基于 checkpoint + state recovery:

每 N step 自动 checkpoint

保存 optimizer state + lr scheduler

支持 resume training

🔥 追问:

DDP 多机断了怎么恢复?

有没有 partial rollback?

Q11: 你说效率提升60%，怎么算的？

指标拆解：

交付周期：3天 → 1天（**pipeline**自动化）

部署时间：1天 → 60min（**one-click deploy**）

人工操作减少：约 60%+

🔥 追问：

有没有 **A/B test**？

是平均值还是**P95**？

哪一环节贡献最大？

Q12: 推理性能指标？

标准回答模板：

QPS: xxx

P95 latency: xxx ms

tokens/s: xxx

GPU utilization: xx%

🔥 追问：

baseline是多少？

提升来自哪里？

Q13: K8s你做了吗？

K8s是底层资源编排系统，由基础设施团队提供，我们的工作主要在其之上做**AI**训练与推理任务的调度抽象层和模型生命周期管理，包括任务封装、资源请求描述、模型部署流水线等。

Q14: 你这个“训推与异构算力平台”整体是解决什么问题的？

这个平台主要解决企业在大模型落地过程中三个核心问题：

第一是模型全生命周期管理碎片化，从数据、训练、评估到部署缺乏统一链路；

第二是异构算力环境复杂，包括**GPU**、昇腾**NPU**、**DCU**，不同框架（**CUDA/CANN**）导致模型迁移成本高；

第三是推理与训练系统割裂，训练与推理各自独立，缺乏统一调度与模型资产管理能力。

因此我们构建了一个统一平台，把数据、训练、推理和部署统一成一个闭环系统。

Q15: 你这个平台整体架构是怎么设计的？

API层（FastAPI）：统一对外提供训练、推理、模型管理接口

调度层（Task Orchestration）：管理训练任务、推理任务生命周期

模型层（Model Registry）：管理**LLM/embedding/reranker**版本

执行层（Execution Engine）：对接 **vLLM / SGLang / MindIE**

资源层（K8s + GPU/NPU/DCU）：负责资源调度与容器运行

Q16: 调度层你们自己做了吗？还是K8s？

K8s负责底层容器与资源调度，我们在其上做的是AI任务调度抽象层：

包括：

训练任务 DAG 编排

模型依赖关系管理

GPU显存/算力感知调度策略

推理服务生命周期控制

Q17: 你觉得你这个项目最大的技术难点是什么？

最大难点不是单点技术，而是：

👉 “异构算力 + 多模型 + 多推理引擎”的统一抽象

核心挑战：

不同框架语义不一致

推理路径复杂

性能优化需要跨层协同

Q17:

Q17:

B. 通信领域通用大模型训练与对齐优化

简历版

项目名称：通信领域通用大模型训练与对齐优化

技术栈：PyTorch、QLoRA、LoRA、Post-Train、SFT、DPO、分布式训练、模型评测体系

项目概述：基于开源大模型构建通信行业专属大模型训练体系，完成Post-Train持续预训练、SFT监督微调及DPO偏好对齐全流程研发；搭建数十GB通信领域语料库与多任务指令数据集，采用 QLoRA+DeepSpeed 实现低成本训练；构建专业评测体系与数据闭环优化机制，最终通信场景问答准确率提升15%，幻觉率下降7%，模型成功落地智能运维、故障诊断、招投标分析等多个生产场景。

核心职责：

1. 搭建通信领域 Post-Train → SFT → DPO 完整训练流水线；

2. 构建数十GB领域语料、数万条 SFT 数据及偏好对齐数据集；

3. 基于 QLoRA+DeepSpeed 实现高效增量训练，显著降低训练成本；

4. 建立自动评测与数据回流闭环，持续优化模型专业能力与稳定性。

****项目名称**:** 通信领域通用大模型训练与对齐优化

****技术栈:** **PyTorch、QLoRA、LoRA、Post-Train、SFT、DPO、分布式训练、模型评测体系

****项目概述**:** 针对通信行业算网运维、通信故障诊断、招投标分析、财务稽核等核心场景，基于 Qwen2.5-72B 开展领域化训练与能力对齐，构建覆盖“领域知识注入→指令理解→偏好对齐”的全流程训练体系；最终相较于原生开源底座模型，业务场景问答准确率提升15%，模型幻觉率降低7%，故障排查场景任务完成率提升12%。

****核心职责**:**

1. ****数据集构建****: 整合行业文档、历史故障工单、招投标资料、运维手册等内外部私有数据，分别构建面向领域知识注入的 **Post-train** 无标注语料，高质量通信专属 **SFT** 训练数据集，以及构建通信领域偏好 **DPO** 数据集；`设计完整数据处理流水线，完成文本解析(PDF/word/HTML等异构文档解析)、格式统一、脏数据过滤、相似度去重(MinHash+Embedding相似度去重)、业务数据脱敏(PII敏感信息脱敏)、高质量样本筛选等工作。`

2. 通信领域持续预训练 (Post-Train): 构建通信专业语料训练集、实现领域增量训练Pipeline、设计学习率warmup与Cosine Decay策略、采用QLoRA + DeepSpeed降低训练成本、监控Loss、Perplexity等训练。对比实验: LoRA 和 全量参数

3. 多阶段监督微调 (SFT): 覆盖多个任务: 故障定位问答、运维工单分析、CLI指令解析、招投标信息抽取、财务规则审核、专业知识问答。第一阶段: 增强指令跟随、格式遵循和多轮对话能力; 第二阶段: 重点强化通信术语理解、故障分析推理和专业知识生成能力。

4. 强化学习对齐 (DPO): 设计评价维度: 专业准确性、事实一致性、推理完整性、回答可执行性、幻觉控制

5. ****模型评测与迭代****：搭建通信领域专项评测体系。通用能力：BLEU、ROUGE、MT-Bench；专业能力：通信知识问答准确率、故障定位成功率、工单解析准确率、招投标信息抽取准确率；安全与稳定性：Hallucination Rate、Consistency Score、Refusal Rate。建立：数据分析、模型训练、自动评测、错误归因、数据优化、重新训练的持续迭代优化闭环。

6. 落地赋能：**将成型通信大模型无缝接入公司智能体开发平台，实现垂直模型多业务场景复用。

基于开源大模型构建通信行业专属大模型训练体系，完成Post-Train持续预训练、SFT监督微调及DPO偏好对齐全流程研发；搭建数十GB通信领域语料库与多任务指令数据集，采用QLoRA+DeepSpeed实现低成本训练；构建专业评测体系与数据闭环优化机制，最终通信场景问答准确率提升15%，幻觉率下降7%，模型成功落地智能运维、故障诊断、招投标分析等多个生产场景。

数据类型	规模	用途
Post-Train语料	数十GB	领域知识注入
SFT指令数据	数万条	业务能力训练
Preference数据	数千组	偏好对齐训练

模拟问题

Q1: 请先介绍一下你这个项目，重点讲清楚业务背景、目标以及你负责的部分。

这个项目的目标是构建通信领域专属大模型，覆盖算网运维、通信故障诊断、招投标分析、财务稽核等多个业务场景。

项目启动时存在三个核心问题：

通用大模型缺乏通信领域知识，对于专业术语和故障场景理解不足：

对通信运维流程缺乏认知，回答容易出现幻觉；

无法满足企业内部业务要求，例如工单分析、招投标审核等专业任务。

因此我们设计了：

领域数据构建



Post-Train领域知识注入



SFT任务能力训练



DPO偏好对齐



专项评测体系



持续迭代优化

我主要负责：

数据集建设全流程

Post-Train训练Pipeline

SFT训练体系设计

DPO偏好数据构建

评测体系建设

模型迭代优化

Q2：为什么要采用 Post-Train + SFT + DPO 三阶段训练，而不是直接SFT？

因为三者解决的问题不同。

Post-Train（属于知识缺失问题）

解决：模型不知道通信知识

例如：BBU是什么？AAU是什么？OTN和PTN区别？

SFT（属于任务能力问题）

解决：知道知识，但不会按照任务要求使用

例如：根据告警日志定位故障，按照JSON格式输出

DPO（DPO负责学习偏好）

解决：多个答案都正确，哪个更符合专家偏好

例如：

故障分析：

答案A：

检查光功率

答案B：

先检查告警等级

再检查链路状态

最后验证光功率

显然B更符合专家经验。

Q3：为什么选择DPO而不是RLHF？

原因1：实现复杂度

RLHF需要：

SFT → Reward Model → PPO

训练链路长。

工程复杂。

原因2：训练稳定性

PPO经常出现：

Reward Hacking

KL Collapse

梯度震荡

训练成本高。

原因3：数据现状

已经有大量：

Chosen, Rejected

偏好数据。

Q4：DPO核心公式是什么？

Q5：为什么不用ORPO或者SimPO？

我们做过实验。

ORPO

优点：

简单

资源消耗小

缺点：

专业场景提升有限

SimPO

优点：

训练更稳定

缺点：

对高质量偏好数据要求更高

Q6：通信数据是怎么构建的？

主要来源：

运维手册

历史工单

故障案例库

招投标文件

规范标准

FAQ知识库

数据处理流程：

文档解析

↓

清洗

↓

去重

↓

脱敏

↓

质量评估

↓

训练集

Q7: 为什么使用MinHash+Embedding双重去重?

因为单纯MinHash无法解决语义重复。

第一层: MinHash, 速度快。

第二层: Embedding, 计算向量相似度。

Q8: Embedding阈值怎么定?

测试:

0.80

0.85

0.90

0.92

0.95

随机抽取: 人工判断是否重复。

Q9: 为什么选择QLoRA而不是全量微调?

核心原因: 成本

Q10: QLoRA为什么不会明显掉点?

原因是:

训练时:

基础权重冻结

但前向传播仍使用完整知识。

实际更新的是:

$\Delta W = BA$

低秩矩阵。

对于通信场景:

知识迁移量有限。

LoRA已经足够。

Q11: 训练过程中遇到过什么线上问题?

问题1: 灾难性遗忘

通信能力提升, 通用能力下降

原因:

领域数据占比过高

解决:

加入20%通用数据混训

问题2: 格式幻觉

要求JSON输出。模型输出: 解释文字+JSON

解决:

增加格式约束SFT样本

并引入:

Format Reward

问题3: 故障分析逻辑跳步

直接输出结论。没有排查过程。

解决：

加入：CoT数据

例如：告警分析 → 链路分析 → 设备分析 → 故障定位

Q12：问答准确率提升15%，这个指标怎么测出来的？

构建独立测试集：

通信知识问答，2000条，专家人工标注标准答案。

评测方式：

LLM Judge + 人工抽检

评分维度：

事实正确性

专业术语准确性

逻辑完整性

Q13：如果让你重新做一次，你觉得最大的优化点是什么？

第一：RAG增强训练

指在训练或微调大模型本身时，利用RAG技术动态引入外部知识，来生成更高质量的训练数据。例如：先通过RAG检索相关信息，再让模型基于这些信息生成带引用的回答，然后用这些合成数据去微调模型。这能提升模型的事实性和引用能力。

第二：Agent能力训练

Q14：为什么选择Qwen2.5-32B而不是7B、14B、72B？

主要从效果、成本和部署三个维度考虑。

从32B到72B：参数增加2倍，效果只提升2%，收益明显递减。

Q15：你说收益递减，依据是什么？

参考Scaling Law。

模型能力近似满足：

$$L(N) = AN^{-\alpha} + B$$

其中：

N = 参数量

L = Loss (Loss 越小，模型能力、精度、效果越好)

随着参数增加：

Loss下降越来越慢

我们判断收益递减，主要依据大模型缩放定律。模型损失 Loss 和参数量满足幂函数关系，参数量越大，Loss 下降速度越慢。结合通信场景实测：7B 到 14B、14B 到 32B 参数扩容收益都很可观，但从 32B 提升到 72B 后，效果提升明显放缓，因此判定已经进入边际收益递减区间。

Q16: Post-Train学习率怎么设置?

Post-Train属于继续预训练，不是从头训练。

学习率过大会：覆盖原有知识（灾难性遗忘）

采用：Cosine Decay + Warmup

学习率：2e-5（通过实验所得）

warmup: 3%

Q17: 怎么看训练发散?

观察Loss曲线

监控：Gradient Norm

Q18: Post-Train阶段主要看哪些指标?

- 1 Loss: 训练是否正常。
- 2 Perplexity: 语言建模能力。
- 3 通信专项评测：每隔500 step，跑专项Benchmark。

Q19: Checkpoint如何选择?

Loss最低 $\neq$ 效果最好

every 1000 step 保存，并测：

通信QA

故障诊断

工单分析

综合打分。

Q20: 为什么做两阶段SFT?

阶段1: 通用指令能力。

训练：

指令跟随

格式遵循

多轮对话

阶段2: 专业能力。

训练：

通信知识

故障推理

工单分析

如果直接混训：

模型容易：格式能力不足 或者 专业能力不足

Q21: SFT学习率为什么比Post-Train小?

1e-5，实际上SFT更容易过拟合。
SFT数据量：几十万
远小于：Post-Train数十亿Token

学习率过大：快速记忆样本，泛化能力下降。

Q22: SFT训练多久停止?

采用Early Stop。
监控: Validation Loss

Q23: DPO训练最重要的参数是什么?

β (beta)，DPO核心超参。
控制：偏好学习强度

β	效果	
----	---	
0.01	学不动	
0.05	较好	
0.1	最优	
0.5	过拟合	

Q24: DPO Loss下降是否代表效果提升?

不一定。
经常出现：DPO Loss下降
但是：幻觉率上升

同步监控：

- Hallucination Rate
- Consistency
- Task Accuracy

Q25: Qwen2.5-32B训练为什么选择ZeRO-3?

FP16下: 32B × 2Byte ≈ 64GB

训练时还有:
Optimizer
Gradient
Activation

实际: >200GB

ZeRO-1

- 仅切Optimizer
- 显存不足。

ZeRO-2

- 切Optimizer+Gradient
- 仍然不足。

ZeRO-3

切：

Parameter

Gradient

Optimizer

全部分片。

最适合32B训练。

Q26: ZeRO-3缺点是什么?

通信开销大。

频繁：

All Gather

Reduce Scatter

Q27: 梯度累积怎么算?

公式：

Global Batch = Micro Batch × GPU数 × Gradient Accumulation

例如：

Micro Batch = 2

GPU = 8

GA = 16

那么：

Global Batch=2×8×16=256

Q28: 如何提升Token利用率?

很多训练资源浪费在Padding。

Packing，把多个短样本拼接。

50+100+200+300

凑满：2048窗口

Q29: 如果老板只给你8张A800，你怎么训练Qwen2.5-32B?

QLoRA

+

ZeRO-3

+

Gradient Checkpoint

+

FlashAttention2

+

Sequence Packing

Q30：基于 Qwen3-32B 模型预训练，在128 张 A100 上训练，预训练数据为 15B 中英文数据（5%的 32k长文本）， epoch设置为3，最快训练需要多久？

最快约 10 天级别完成 pretraining，工程现实通常在 12-18 天之间。

Q31：如何评估预训练的效果？

1、查看训练指标

现象	含义
train ↓, val ↓	正常学习
train ↓, val plateau	数据或容量瓶颈
train ↓, val ↑	过拟合 / 数据污染
loss 振荡	学习率 / batch / 并行问题

2、评估领域知识正确性

通信知识QA，查看 Accuracy（单选/多选）、EM（Exact Match）、F1（结构化答案）

协议理解能力

3、RAG能力

RAG评估指标（Retrieval Accuracy、Faithfulness、Citation Accuracy）

4、工程问题解决能力

故障诊断能力、KPI分析能力、网络优化建议质量等

5、推理能力（通信特化）

Multi-step reasoning、Constraint reasoning（约束推理）、Counterfactual robustness（反事实鲁棒性）

6、业务可用性评估

人类专家评估、在线模拟、A/B系统测试（故障解决时间下降、工单解决准确率上升、工程师工作量下降）

7、设计专用通信评测数据集

通信知识集、KPI分析集、故障诊断集、运维建议集、信令流程集

Q32: 通信数据“脏 + 噪声 + 模板化严重”，如何解决

1) 分层数据清洗

rule-based filter (日志模板去重)

MinHash / SimHash 去重

heuristic 过滤 (低信息密度文本)

2) 通信语料“结构化重写”

3) 重采样

提升高价值数据权重: (数据分类打标 + 赋权, 离线加权重采样 (构造训练集))

故障诊断日志 ↑↑

KPI 因果分析 ↑↑

3GPP 原文 ↓ (降权)

Q33: loss下降但能力不提升, 如何解决?

1) 分domain loss监控

必须拆:

Chinese telecom loss

English spec loss

KPI logs loss

code/log structured loss

👉 防止单域过拟合

2) 增加“对比学习信号” (隐式)

构造:

正确故障分析 vs 错误分析

3) 学习率 warmup (预热) 拉长

32B 通信数据:

通常需要更长 warmup (2% → 5% tokens)

防止 early collapse

Q34: 长文本能力退化, 如何解决?

1) 长样本“强化采样” (重复采样)

不是5%, 而是:

token-level oversampling (不是 sample-level)

👉 让32K token贡献 ≥ 15%

2) context packing优化

避免:

多段无关拼接

改成:

KPI 时间序列连续拼接

单故障完整链路拼接

3) RoPE scaling校准

常用:

NTK-aware scaling

YaRN (推荐)

👉 防止32K attention collapse

4) long-short curriculum learning

训练顺序:

4K → 8K → 16K → 32K

Q35: 幻觉严重

1) RAG强绑定训练

训练时加入:

retrieved context only answer loss (训练时, 只允许模型看检索到的“小抄”来回答, 不看小抄答对的统统不算分, 甚至要扣分!)

evidence grounding loss (不仅要答对, 答出来的每一个字, 都必须能在检索到的原文里找到明确出处(证据链)。)

2) 加入“不可回答样本”

例如:

Q: 该问题是否可以从给定日志判断?

A: 不确定 / 信息不

👉 降低“强行回答倾向”

3) fact-consistency reward (如果有后训练)

Q36: 长上下文能力是通过继续预训练自然获得的, 还是单独做了 long-context adaptation (比如 RoPE scaling / NTK / YaRN) ?

工业界基本是“混合方案”, 不是二选一

可以理解为三段式:

模型结构层(必须): 扩展 **position encoding** 上限

训练层(关键): 注入长依赖分布

对齐层(最后): 让模型学会“正确用长上下文”

长上下文能力一般不是单靠 **continued pretraining** 自然获得的, 而是一个“结构改造 + 数据课程学习 + 对齐训练”的组合过程。

如果只做 **continued pretrain**, 在 **RoPE** 或位置编码没有处理的情况下, 模型会在 **extrapolation** 阶段出现 **attention distortion**, 远距离 **token** 的有效性会显著下降。

因此工业界通常会先做 **RoPE scaling / NTK** 或 **YaRN** 这类 **position interpolation** 方法, 把 **attention** 的频域稳定性扩展到目标 **context length**, 然后再通过 **curriculum learning** 的方式逐步引入长序列数据, 比如从 **4K → 16K → 32K**。

最后还会做专门的 **long-context task alignment**, 比如 **multi-document QA** 或 **needle retrieval**, 来避免“能看长文本但不会用”的问题。

Q37: 在长上下文数据占比较高的情况下, 你们是如何处理 short/long sequence 的训练冲突的? 是混训还是分阶段训练?

short/long sequence 的冲突 = 梯度贡献不均 + 计算效率不均 + 表示学习目标不一致
“分阶段训练 + **batch-level mixture control** (动态混合)”的组合方案

1、分阶段训练 (**2K → 4K → 8K → 16K → 32K**)

2、**Token-level mixture** (控制 **short token/long token** = 2:1)

3、**Dynamic batching** (按长度分桶, 再按桶取batch)

4、**Loss reweighting** (降低 **long token** 的权重)

五、一个具体例子（帮助你彻底理解）

假设你设定：

类型 token target

short 70%

long 30%

Step 1: Token-level mixture

系统先决定：

这个 step 需要：

- 14K short tokens

- 6K long tokens

👉 注意：这是 token budget，不是 sample 数

Step 2: 进入 dynamic batching

short tokens:

[200, 300, 500, 800]

[400, 600, 700]

long tokens:

[12K]

[18K]

Step 3: batch组装

dynamic batching 会：

short → 组成高效 batch（padding低）

long → 单独 batch（避免拖慢）

Q38：你们在预训练或微调阶段，是如何定义数据质量指标的？有没有做过 data mixture reweighting 或 curriculum strategy？

Q39：你们评估 long-context 能力时，主要依赖 needle-in-a-haystack 还是 RULER / passkey retrieval？有没有发现训练指标和真实效果不一致的情况？

needle-in-a-haystack + RULER + passkey retrieval + 真实任务评测（RAG/多文档QA）组合使用

1、在超长上下文中插入一个“关键事实”，看模型能否找到。

2、类似 needle，但更结构化：多个 key-value pairs，不同位置插入，要求精确 recall

3、长上下文中的综合推理能力：retrieval、multi-hop reasoning、distractor filtering、long instruction following

Q40：RoPE scaling / NTK / YaRN

目前YaRN最常用

<https://www.doubao.com/chat/38432200367245826>

Q41：

Q42：

C. 企业级智能体平台

简历版

项目名称：企业级智能体研发平台

技术栈：Python、LLM、Multi-Agent、RAG、Milvus、ASR/TTS、评测算法

项目描述：面向企业客户的低代码智能体开发平台，支持任务拆解、工具调用与检索增强生成（RAG）。本人负责LLM能力增强与RAG评测体系建设。

核心工作：

- LLM与检索优化：设计多层级RAG架构，实现稀疏与稠密多路召回；结合Prompt优化、答案融合与推理拆解降低模型幻觉。在测试集评估中，幻觉率降低约19%，回答准确率提升约25%。
- 自动化评测构建：搭建RAG自动化评测模块，围绕忠实度、答案相关性、上下文精度/召回率及实体召回率进行全链路评估，替代传统人工抽检，降低约60%的评估人力成本。
- 语音能力集成：联动ASR/TTS语音模型，实现多终端语音交互功能，提升了平台在企业级交付场景中的适配能力。

模拟面试

****项目名称：企业级智能体研发平台****

****技术栈**：**Python、LLM、Multi-Agent、RAG、Milvus向量库、ASR/TTS、评测算法

****项目描述**：**面向企业级客户打造低代码智能体开发平台，支持任务拆解、工具调用、自主决策、检索增强生成能力，助力业务快速搭建专属自动化智能体；本人负责LLM能力增强与RAG全维度评测体系搭建两大核心模块。

****核心工作**：**

1、LLM能力增强。优化多层级RAG检索架构，融合稀疏检索、稠密检索多路召回策略；搭配Prompt优化、答案融合、推理拆解等技术，有效降低模型幻觉，提升复杂问题回答准确率；优化后知识库问答幻觉率下降19%，答案整体准确率提升25%。

2、RAG评估功能开发。搭建自动化评测模块，围绕**忠实度、答案相关性、上下文精度/召回率、实体召回率**五大核心指标，完成全链路自动化评测；替代传统人工抽检模式，降低60%以上RAG评估成本；

3、平台能力适配：联动语音模型，实现智能体语音交互能力，适配多终端使用场景，完善平台企业级交付能力。

模拟问题

Q1：请你先整体介绍一下这个“企业级智能体研发平台”的架构设计？

这个平台整体是一个面向企业的低代码智能体开发系统，核心目标是让业务人员可以快速构建具备工具调用、RAG检索、多轮推理能力的智能体。

整体链路如下：

用户请求入口层

- 支持文本/语音输入（ASR）
- 请求进入 API Gateway（Python + FastAPI）

智能体编排层（Multi-Agent Orchestrator）

- 根据任务类型进行 agent routing
- 支持 planner agent + executor agent 架构
- 进行任务拆解与子任务分发

知识增强层（RAG）

多路召回：

稀疏检索（BM25）

稠密检索（Embedding + Milvus）

rerank + context compression

LLM推理层

Prompt template + tool calling

支持 function calling / ReAct

工具执行层

外部API调用 / 数据库 / 企业系统

输出层

文本输出 / TTS语音输出

Q2：为什么不直接用纯向量检索？

纯向量检索在“语义相似但关键词缺失”的情况下容易召回错误内容

在企业知识库中存在大量：

产品型号

编号

规范条款

→ 这些对关键词非常敏感，纯 embedding 会丢失精确匹配能力

Q3：稀疏检索在你们场景中解决了什么问题？

BM25用于保证“精确匹配召回”

特别是在：

工单号

设备编号

法规条款

提升 recall@k 的稳定性

Q4：Milvus在你们系统里承担什么角色？它的瓶颈怎么处理？

Milvus负责向量索引（HNSW/IVF_FLAT）

优化点：

IVF参数调优（nlist / nprobe）

分库分索引（按业务域拆分collection）

热数据cache embedding

瓶颈主要在：

高并发 query latency

解决方式：

batch query

embedding cache

限制topK + rerank前置过滤

Q5: “忠实度”你是怎么定义和计算的?

Faithfulness = 答案是否可以被检索上下文支持

方法:

将答案拆成 **claim-level statements**

用 **NLI** 或 **LLM judge** 判断每个**claim**是否被**context entailed**

Q6: 这些指标是人工标注还是自动评估? 如果是自动评估, 如何保证可信度?

采用“**LLM-as-Judge + rule-based hybrid**”

其中:

LLM判断语义一致性

rule用于实体/数值对齐

为降低偏差:

引入少量人工**gold set** (约200条)

用来校准**LLM**评分偏置

Q7: 你说降低幻觉率19%, 这个“幻觉率”具体怎么算?

定义:

hallucination = 无法被**context**支持的**claim**数 / 总**claim**数

在测试集上:

baseline: 28%

优化后: 9%

→ 下降约19%

Q8: 线上有没有出现过RAG“检索正确但答案仍然错”的情况? 怎么定位的?

检索正确但答案错误的问题

确实出现过

root cause:

context中信息冲突 (多个版本知识)

LLM没有做冲突消解

解决:

加入 **context ranking** (时间+权重)

prompt加入“**conflict resolution instruction**”

Q9: 系统延迟怎么控制? Multi-Agent + RAG + rerank 不会很慢吗?

延迟优化

原始链路:

RAG + rerank + agent planning → 2.8s

优化后:

embedding cache

并行召回 (BM25 + vector async)

rerank top50→top10

→ 降到 1.4s

Q10: 如果现在让你再优化一版, 你会动哪一层?

我会重点优化三点:

Agent planner (减少不必要的多轮推理)

reranker轻量化 (蒸馏模型替代cross-encoder)

引入 speculative retrieval (提前预取知识)

Q11: 你们这个系统到底是“智能体平台”, 还是只是 RAG + prompt + tool call 的工作流编排? 和 LangChain / Dify 有什么本质区别?

客观来说, 我们的系统早期确实是 workflow-based agent (工作流型智能体), 但在后续迭代中逐步增强了“决策层”。

1) 基础层 (确实类似 LangChain)

RAG 检索 (BM25 + 向量)

Prompt 拼装

Tool calling 执行

👉 这一层确实和 LangChain / Dify 很接近

2) 增强层 (我们做的差异点)

我们引入了任务级规划器 (Planner Agent):

不是简单顺序 workflow

而是:

任务拆解 (decomposition)

子目标排序 (ordering)

是否调用工具的决策 (tool policy)

👉 这一点是区别点之一

3) 平台层差异 (真正工程差异)

我们不是“单个 agent runtime”, 而是:

多 agent registry (不同业务 agent)

动态 routing (根据任务选择 agent)

RAG + tool + voice 多模态统一编排

🏆 客观承认边界

但我也必须说明:

当前版本仍然不是完全 autonomous agent (没有长期自学习闭环)

Q12: BM25 和向量召回冲突怎么办? 权重怎么定? 是不是拍脑袋?

1. 基于置信度/分数的加权融合

核心思想: 不同召回通道对同一信息有不同置信度 (如相似度分数、BM25分数、图谱关系强度)。优先采信置信度最高的结果。

2. 基于时间/版本的新鲜度优先

核心思想: 冲突往往因信息过时引起, 默认采用更新时间更近的答案。

3. 基于来源权威性的层级优先

核心思想: 不同来源的可信度不同 (如官方文档 > 社区博客 > 自动抽取)。

4. 基于一致性检验的共识提取

核心思想: 如果大多数召回通路支持同一事实, 该事实更可能正确。

5. LLM作为裁判的语义裁决

核心思想: 将冲突片段及各自的来源、分数、时间等上下文提交给LLM, 由LLM依据逻辑和常识判断哪个更合理。

6. 避免融合: 分情景返回

核心思想: 如果冲突无法可靠解决, 不如坦陈不确定性, 让下游或用户决策。

Q13: 幻觉率这个指标是不是你自己定义的? LLM-as-Judge会不会自嗨?

1) 幻觉率定义是可解释的, 而不是随意定义, 我们定义的是:

hallucination = claim-level unsupported ratio

2) LLM-as-Judge确实有bias问题

(1) 模型隔离

Judge model ≠ generation model, 使用不同模型 (避免 **self-bias**)

(2) 规则约束辅助

数值一致性 **rule check, entity match check**

(3) 人工校准集

200条 gold set, 用来校准评分分布 (不是直接训练, 只做 **bias correction**)

Q14: 为什么不用 LangChain / Dify 直接做?

1) 企业级定制能力

我们不是做 **demo agent**, 而是:

多知识域隔离

多租户隔离

agent policy 可控

👉 **Dify / LangChain** 更偏通用框架

2) 评测体系 (核心差异点)

我们有完整 **RAG evaluation pipeline**:

faithfulness

recall

entity accuracy

answer relevance

👉 这一点是很多框架没有内建的

3) 语音 + agent 融合链路

我们是:

ASR → agent → RAG → TTS

而不是纯文本链路

Q15: 你说延迟从 2.8s 降到 1.4s, 你具体怎么 profiling 的? 有没有证据链?

1) 我们先做了 tracing (不是猜)

embedding latency

retrieval latency

rerank latency

LLM latency

tool latency

2) 真实瓶颈定位结果

rerank + context packing 占比最高 (约 40%)

3) 优化手段

(1) rerank优化

top50 → top10 rerank

轻量cross encoder替换

(2) 并行化

BM25 + vector async

tool prefetch

(3) cache

embedding cache (query-level + chunk-level)

4) 关于“证据链”

tracing log

p95 latency before/after

stage breakdown dashboard

不是单点优化, 是可观测系统优化

Q16:

6.2 比赛经历

2026-FlagOS开放计算全球大赛

简历版

FlagOS开放计算全球大赛 | 国家级行业赛事 | 2026.06 | 队长 | 三等奖

•

基于Qwen3-4B模型与ICL范式, 优化Nanobot智能体运行架构, 设计自适应少样本遴选策略, 解决长上下文场景资源浪费、标注精度偏低等问题; 引入轻量化GRPO强化学习方案, 零参数更新优化模型判别能力。

•

覆盖情绪检测、主题分类、百科问答、算子代码生成多类任务, 搭建代码闭环核验体系与多线程标注框架, 多维度优化算法策略并完成消融实验, 沉淀可落地的长文本数据标注解决方案。

模拟面试

****赛题：**[\[长上下文场景中大模型自动数据标注\]](https://flagos.io/RaceDetail?id=296fmsd8&lang=cn)(<https://flagos.io/RaceDetail?id=296fmsd8&lang=cn>)（三等奖）**

****项目描述：****本次挑战赛旨在**探索大语言模型在超长上下文（如数万至数十万token）范式下的数据标注能力提升方法**，重点评估模型在复杂、长序列标注场景中的稳定性、泛化性与实用性。

****技术难点**：**

- 在超长上下文场景下，如何设计有效的模型指令与提示策略，引导 LLM 稳定、高质量地完成数据标注任务？
- 当可用标注示例数量显著超过模型上下文容量时，如何为待标注数据构造信息密集、结构合理的超长上下文输入？
- 在自动多轮对话或持续交互场景中，如何高效利用超长上下文，实现一致性与可扩展性兼顾的数据标注？

项目名称

长上下文场景中大模型自动数据标注

技术栈：

Qwen3-4B、Context Engineering、Prompt Optimization、ICL、Multi-class GRPO、NanoBot、LLM Evaluation、Python

项目背景

针对长上下文（32K~128K Token）自动数据标注场景，传统 Few-shot Prompt 存在上下文利用率低、类别边界模糊、Prompt 依赖人工调优等问题。

项目从：

- Context Engineering（上下文工程）
- Prompt Optimization（提示词优化）
- RL-based Boundary Learning（强化学习边界学习）

三个层面构建自动标注优化框架，实现复杂长序列场景下的高质量自动标注。

最终获得长上下文场景中大模型自动数据标注挑战赛全国三等奖。

核心工作

一、构建任务自适应上下文组织框架

针对长上下文场景中信息密度低、注意力分散的问题，设计任务级 Context Pipeline。

上下文组成

System Instruction

↓

Task Definition Card

↓
Boundary Definition

↓
Dynamic Few-shot Examples

↓
Output Constraint

↓
Target Sample

其中：

Task Definition Card

定义：

- 任务目标
- 标签含义
- 判断原则
- 特殊规则

降低模型理解成本。

Dynamic Few-shot Retrieval

设计动态示例检索机制：

固定 Few-shot：

每次固定选择5个示例

存在：

冗余样本过多
边界样本不足
Token浪费

问题。

因此设计：

Boundary-first Retrieval

优先召回：

- 类别边界样本
- 历史误分类样本
- 高置信代表样本

动态决定：

$k \in [3, 20]$

而非固定数量。

实现有限 Token Budget 下的信息密度最大化。

二、构建 Prompt 自动优化闭环

传统 Prompt 调优依赖人工经验：

修改 Prompt
↓
跑测试集
↓
看结果
↓
继续修改

效率极低。

基于 NanoBot 构建 Prompt Optimization Agent：

Error Collection

收集：

- 错误样本
 - 边界样本
 - 低置信样本
-

Error Analysis

自动分析：

规则缺失
类别描述模糊
示例覆盖不足

等问题。

Prompt Rewrite

自动生成：

- 新规则
- 新约束
- 新示例

形成候选 Prompt。

Regression Evaluation

自动执行：

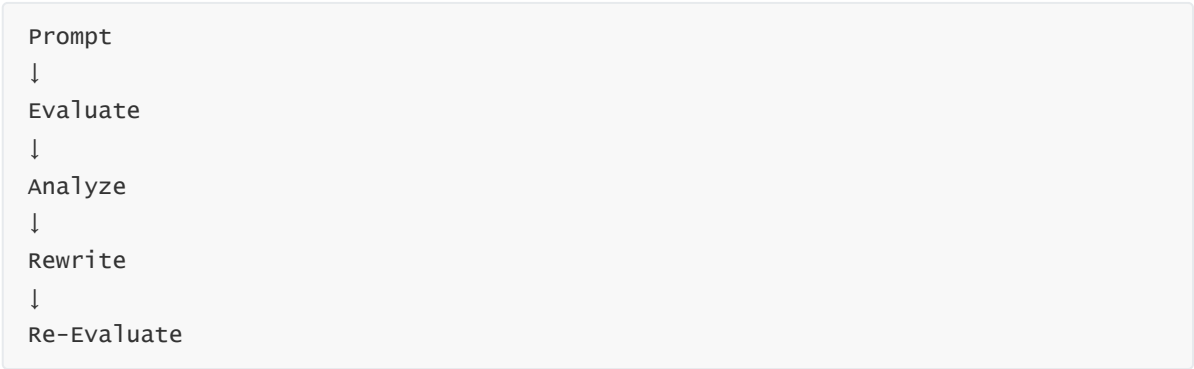
Prompt A/B Test

评估：

- Accuracy
- Macro-F1
- Consistency

指标变化。

形成：



优化闭环。

三、利用 GRPO 学习分类边界

项目最核心创新点。

问题

长上下文分类任务中：

很多错误来自：

- 类别定义不清晰
- 类别边界重叠
- Prompt描述不充分

例如：

故障处理

VS

故障定位

投诉

VS

咨询

需求

VS

建议

边界非常模糊。

数据构建

将误分类样本自动收集：

Prediction $\neq$ Label

构建：

Hard Example Dataset

Multi-class GRPO训练

奖励函数设计：

正确分类：

+1

错误分类：

-1

边界样本正确分类：

+2

重点强化边界学习能力。

Boundary Knowledge Extraction

GRPO训练后：

分析模型学习到的：

类别区分特征
类别判定规则
边界描述

自动生成：

Boundary Definition Card

例如：

咨询：
以获取信息为目的

投诉：
以表达不满并要求处理为目的

Prompt回灌

将边界知识重新注入：

Prompt
Few-shot
Rule Card

实现：

RL Learning
↓
Boundary Discovery
↓
Prompt Enhancement
↓
Inference Improvement

闭环优化。

技术亮点

Context Engineering

- 动态上下文编排
- Boundary-aware Few-shot Retrieval
- Token Budget Optimization
- 长上下文信息密度提升

Prompt Optimization

- NanoBot 自动调优
- 错误驱动 Prompt 演化
- Prompt A/B Evaluation

RL-based Boundary Learning

- Multi-class GRPO
- Hard Example Mining
- Boundary Knowledge Distillation
- Boundary Definition Generation

面试中的一句话总结

这个项目本质上是在长上下文自动标注场景下，构建了一套“Context Engineering + Prompt Optimization + RL-based Boundary Learning”的闭环优化系统。通过动态上下文组织提升信息利用率，通过NanoBot实现Prompt自动优化，再利用Multi-class GRPO从误分类样本中学习类别边界知识并回灌Prompt，最终实现自动标注能力的持续迭代与自进化。

模拟问题

Q1：先简单介绍一下你认为最有代表性的大模型项目，你解决的核心问题是什么？

这个项目是一个长上下文（32K~128K Token）场景下的大模型自动数据标注系统，核心目标是在不使用 embedding、reranker 等外部神经网络模型的前提下，仅依赖 Qwen3-4B，通过 Prompt + Context Engineering + RL 来提升标注质量。

传统 Few-shot 标注在长上下文中存在三个问题：

上下文过长导致注意力分散

类别边界模糊导致误判

Prompt 依赖人工调优，迭代效率低

因此我们从三个方向优化：

Context Engineering：提升信息密度

Prompt Optimization：自动化迭代 Prompt

GRPO：学习类别边界

最终在比赛中获得全国三等奖。

Q2：为什么长上下文场景下 Few-shot 效果会变差？

第一：长上下文存在明显的 Attention Dilution（注意力稀释）。

第二：固定 Few-shot 很难覆盖边界场景。

第三：长上下文会出现 Lost-in-the-Middle(开头/结尾,信息利用率最高)。

Q3: Lost-in-the-Middle你们是怎么验证的?

我们做过位置消融实验。
固定 Prompt 内容：
 Prompt 内容完全一致
仅调整 Few-shot 位置：
 Head
 Middle
 Tail

Q4: 为什么不用 RAG?

不让使用 embedding，无法获取语义相似度

Q5: 你们怎么做“检索 few-shot 示例”的？完全不用向量模型吗？

Rule-based + Label-aware Selection

具体做法：
 基于标签分桶 (label bucket)
 基于历史误分类样本池 (error memory)
 基于规则匹配 (关键词 + 模板匹配)
 Boundary-first sampling (优先选“模型曾经错过的样本”。)

Q6: 没有 embedding，你怎么判断“边界样本”？不会很主观吗？

我们不是语义相似度判断，而是用行为信号定义边界样本。
边界样本定义为：

模型历史上：
 1. 预测错误
 2. 或置信度低 (logprob proxy)
 3. 或多次预测不稳定

Q7: 你说你做了 Context Pipeline，这一套到底解决了什么问题？为什么普通 Prompt 不行？

普通 Prompt 在长上下文中有两个核心问题：

- 1 信息衰减
 early token 被弱化
 instruction 被覆盖
- 2 结构混乱
 few-shot 示例和规则混在一起，模型无法区分优先级

我们 Context Pipeline 做了结构化：

System
↓
Task Card
↓
Boundary Definition
↓
Few-shot
↓
Constraint
↓

Query

让模型“按层理解任务”，而不是混读

Q8: 为什么 Task Definition Card 能提升效果？它本质不还是 prompt 吗？

👉 降低认知负担 (Cognitive Compression)

我们做了三件事：

把任务拆成固定 **schema**

每个 **schema** 是短句 + 强约束

避免自由描述

Q9: Token Budget Optimization, 具体怎么控制 $k \in [3, 20]$?

我们不是固定 k ，而是动态计算：

$k = \min(\text{max_score_samples} + \text{boundary_samples}, \text{budget} / \text{avg_token_length})$

策略：

budget = 模型最大 **context** - **system** - **query**

boundary samples 优先占位

剩余填高置信样本

Q10: 你说你做了 Multi-Class GRPO, 它和普通 GRPO 本质区别是什么？

普通 GRPO 是在单样本维度做 group relative optimization:

同一个 **prompt**

多个 **sampling trajectory**

做 **reward** 相对比较

👉 本质是: sample-level ranking learning

而我们提出的 Multi-Class GRPO 是：

👉 batch-level + multi-class constraint 的 GRPO 变体

核心变化有三点：

1 **batch** 内包含多个类别（不是单任务 group）

Batch = {class1, class2, class3 ...}

而不是：

Batch = same prompt multiple outputs

2 引入跨类别边界竞争机制

我们不只是比较“同类输出好坏”，而是：

👉 跨类别之间也参与 **reward competition**

例如：

“投诉 vs 咨询”

“故障定位 vs 故障处理”

模型必须在 **batch** 内做结构性区分。

3 **reward** 控制“可更新经验数量”

我们设计了一个关键机制：

reward $\uparrow$ $\rightarrow$ 可更新经验 $\downarrow$

reward $\downarrow$ $\rightarrow$ 可更新经验 $\uparrow$

直觉是：

高 reward: 说明当前 batch 已经稳定 → 不需要大幅更新
低 reward: 说明边界不清 → 加大经验更新力度

Q11: 你这个“reward 控制更新经验数量”具体是什么意思？听起来不像标准 RL

确实不是标准 RL 设计，我们做的是：

👉 把 GRPO 从“优化目标函数”改成“控制学习节奏的机制”

具体实现是：

Step1: 计算 batch reward score

$R_{\text{batch}} = \text{mean}(\text{correct samples}) - \text{mean}(\text{error samples})$

Step2: 映射 update quota

$\text{update_ratio} = \text{sigmoid}(\alpha - R_{\text{batch}})$

Step3: 控制 replay buffer 更新量

reward 高 → 只更新少量 hard samples

reward 低 → 扩大 error samples 回灌

👉 本质是：

reward 不只是“优化信号”，还是“经验流动控制器”

Q12: 你说 batch 内跨类别比较，但不同类别本身语义不同，怎么保证可比性？

👉 比较“分类决策一致性”而不是语义相似性

跨类别比较的不是“内容”，而是：

👉 分类边界清晰度

例如：

咨询 vs 投诉

是一个 decision boundary problem

👉 Multi-Class GRPO 本质不是跨语义比较

👉 而是跨 decision boundary 的结构优化

Q13: 你说 batch reward 控制经验更新，那如果 batch 全对或者全错怎么办？

GRPO 只在“部分正确但存在边界模糊”时才有效

Q14: 你这个机制听起来像“训练控制策略”，那它和 curriculum learning 有什么区别？

确实有相似性，但有本质区别：

Curriculum Learning:

预定义难度顺序

静态 schedule

Multi-Class GRPO:

👉 动态结构学习

区别在三点：

1 difficulty 不是预设的

而是：

$\text{difficulty} = \text{model error} + \text{boundary instability}$

2 batch 内是竞争结构

不是“简单排序”，而是：

👉 class-level competition

3 reward 控制的是“经验流动”
不是 sample selection, 而是:
experience update rate

Q15: 那你这个方法稳定吗? 会不会出现训练震荡?

✗ 问题1: reward oscillation

batch reward 在:

高正确率 ↔ 突然崩溃

解决:

EMA smoothing reward:

由于 batch 内跨类别竞争, reward 波动比较大, 所以我们引入 EMA 对 reward 进行平滑, 减少单 batch 偶然性对训练的影响。

clip update ratio:

为了避免 reward 过低时导致策略更新过激, 我们对 update ratio 做了 clip, 限制每次 policy 更新的最大幅度。

✗ 问题2: class dominance

某些类别长期高 reward

解决:

class-balanced replay buffer:

由于不同类别难度不同, 容易出现 class dominance, 我们用 replay buffer 做类别均衡采样, 防止模型偏向高频或容易类别。

Q16:

2023-全国先进计算技术创新大赛

简历版

全国先进计算技术创新大赛 | 国家级行业赛事 | 2023.12 | 算法核心成员 | 一等奖

- 依托曙光国产软硬件平台, 完成ChatGLM2-6B大模型在DCU硬件与Slurm集群环境下的迁移部署; 运用P-tuning v2微调策略优化模型, 模型整体性能达赛道SOTA级别。
- 完成多场景数据集推理评测, 量化分析模型推理指标偏差原因, 沉淀国产硬件下大模型部署、调优实战经验。

模拟面试

**赛题: [\[2023D3-基于计算硬件平台的AI大模型迁移\]](http://www.ac-innovation.com/#/home) (<http://www.ac-innovation.com/#/home>)
(一等奖)**

项目描述: 基于曙光计算服务平台为基础, 进行大模型迁移或研发。在研发过程中需基于曙光自研的软件栈(配套有成熟软件库、编译器等), 也使用的其他相关软件实现。

核心工作:

- 1、基于国产ChatGLM2-6B中文大模型, 完成了在Slurm调度系统+DCU硬件环境下的环境适配, 并进行P-tuning v2调优, 模型效果达到SOTA水平。
- 2、在推理方面, 模型性能在验证数据集为0.27, 在测试数据集为0.70, 相比于SOTA (ChatGLM2-6B, 0.50) 模型存在一定差距, 通过代码初步分析了相关原因。

模拟问题

Q1: 你们迁移过程中最大的技术难点是什么?

CUDA生态向DCU生态迁移。

原始代码大量依赖:

CUDA

NCCL

Apex

DeepSpeed部分CUDA Kernel

而DCU使用的是: ROCm生态。

因此存在大量兼容性问题。

Q2: 具体遇到了哪些兼容问题?

1 ROCm版本兼容

2 CUDA API不兼容

`torch.cuda.amp --> torch.autocast`

3 NCCL通信问题

`backend='nccl' --> backend='hccl'`

4 自定义Kernel失效

部分FlashAttention实现依赖CUDA Kernel。需要退化到普通Attention实现。

Q3: FlashAttention失效后性能下降多少?

单卡推理吞吐下降约20%~30%。

无法利用FlashAttention的显存优化和Kernel融合能力。

Q4: 为什么没有自己重写Kernel?

比赛周期有限。

重写FlashAttention涉及:

HIP编程

Kernel优化

warp级同步

工作量较大。

Q5: 为什么选择P-Tuning v2?

P-Tuning v2对显存最友好。

Q6: P-Tuning v2原理是什么?

不更新LLM参数。

只训练连续Prompt Embedding。

原输入:

Question

变成:

[P1][P2]...[Pn] Question

其中: P1~Pn 是可学习参数。

训练过程中仅更新Prompt参数。

模型主体冻结。

Q7: Prompt长度设置多少?

实验对比过: 128

Q8: